

Konzeption und Implementierung eines KI-gestützten GitLab-Assistenten zur Aufbereitung von Software-Tickets

Bachelorarbeit

des Studienganges **Informatik**

an der Dualen Hochschule Baden-Württemberg **Stuttgart**

von

Nico Werner

—

Bearbeitungszeitraum	12 Wochen
Matrikelnummer, Kurs	3043145, INF23E
Dualer Partner	Drees & Sommer, Stuttgart
Betreuer*in des Dualen Partners	Denis Günther
Wissenschaftliche Betreuung	—

Ehrenwörtliche Erklärung

Ich versichere hiermit, dass ich die vorliegende Arbeit selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel verwendet habe und diese Arbeit bei keiner anderen Prüfung mit gleichem oder vergleichbarem Inhalt vorgelegt habe und diese bislang nicht veröffentlicht wurde. Des Weiteren versichere ich, dass die eingereichte elektronische Fassung mit der gedruckten Ausfertigung übereinstimmt.

Ort, Datum

Unterschrift

Kurzfassung

Software-Tickets bilden in Entwicklungsteams die Schnittstelle zwischen fachlichem Bedarf, Umsetzung und Prüfung. Relevante Angaben sind häufig unvollständig oder über Beschreibung und Kommentare verteilt. Diese Arbeit untersucht, welche Informationen ein GitLab-Ticket für eine nachvollziehbare Umsetzung und Prüfung benötigt und wie ein KI-gestützter Assistent kontrolliert in einen GitLab-Workflow integriert werden kann. Ziel, Kontext, erwartetes Verhalten, Prüfkriterien und sichtbare Unsicherheit bilden den fachlichen Kern.

Dazu wird Dev-Assist als Proof of Concept in TypeScript mit Express umgesetzt. GitLab-Webhooks lösen die Verarbeitung aus. Der Dienst sammelt Issue- und Kommentardaten sowie optionalen Repository-Kontext, übergibt sie an einen OpenCode-Analyseagenten und prüft die Antwort gegen ein festes Ausgabeschema. Je nach Informationslage entstehen Rückfragen oder ein strukturierter Vorschlag. Das Issue wird erst nach einem gesonderten Publish-Kommando geändert. Ausgabeschema, Schutz vor Bot-Schleifen und lokale Persistenz begrenzen die Autonomie des Modells.

Abstract

Software tickets are the interface between business needs, implementation, and verification in development teams. Relevant information is often incomplete or spread across the description and later comments. This thesis investigates which information a GitLab ticket needs for comprehensible implementation and checking, and how an AI assistant can be integrated into a GitLab workflow under explicit control. Goal, context, expected behaviour, acceptance criteria, and visible uncertainty form the quality core.

Dev-Assist is implemented as a TypeScript and Express proof of concept. GitLab webhooks trigger processing. The service collects issue and comment data plus optional repository context, sends them to an OpenCode analysis agent, and validates the reply against a fixed data contract. Depending on the information available, it asks clarification questions or produces a structured proposal. The issue is changed only after a separate publish command. The data contract, bot-loop protection, and local persistence limit the model's autonomy.

Inhaltsverzeichnis

Ehrenwörtliche Erklärung	I
Kurzfassung	II
Abstract	III
Abbildungsverzeichnis	VII
Tabellenverzeichnis	VIII
Abkürzungsverzeichnis	IX
1. Einleitung	1
1.1. Forschungsdesign	3
1.2. Abgrenzung und Aufbau	3
2. Theoretische und technische Grundlagen	5
2.1. Requirements Engineering und Ticketqualität	5
2.1.1. Normativer Bezugsrahmen und eigene Operationalisierung . .	5
2.1.2. Empirische Befunde zu User Stories und Bug Reports	6
2.2. Automatisierte Qualitätsunterstützung	8
2.3. Aktuelle Forschung zu LLM-gestütztem Requirements Engineering . .	9
2.4. Vergleich der Ansätze und Ableitung des Qualitätsmodells	13
2.5. Large Language Models und agentenbasierte Assistenz	14
2.6. Bewertung generativer Ticketvorschläge	15
2.7. GitLab und technische Integrationsplattform	16
2.8. Risiken und Gestaltungsfolgen	17
2.9. Zwischenfazit	18
3. Anforderungsanalyse	19
3.1. Vorgehen und Nachverfolgbarkeit	19
3.2. Ausgangs- und Zielprozess	20
3.3. Stakeholder, Priorisierung und Nachverfolgbarkeit	21
3.4. Qualitätsmodell für Ticketvorschläge	22
3.5. Funktionale Anforderungen	23
3.6. Nicht-funktionale Anforderungen	24
3.7. Abgrenzung und Zwischenfazit	25
4. Konzeption der Systemarchitektur	26
4.1. Architekturziele und Komponenten	26

4.2.	Bewertung zentraler Architekturentscheidungen	30
4.3.	Ereignis- und Datenfluss	31
4.4.	Prompt, Ausgabeschema und Kontextgrenzen	34
4.5.	Sicherheit, Robustheit und Testbarkeit	35
4.6.	Zwischenfazit	35
5.	Implementierung	36
5.1.	Funktionierender Kern	36
5.2.	Projektaufbau und Laufzeitkonfiguration	37
5.3.	HTTP- und Webhook-Verarbeitung	38
5.4.	GitLab-Client und Prozesssteuerung	39
5.5.	Durchgängiger Verarbeitungsfall	40
5.6.	Zustand und Konsistenz an den Prozessgrenzen	41
5.7.	OpenCode, Prompt und Laufzeitvalidierung	42
5.8.	Kontext, Publish und Repository-Zusammenfassung	43
5.9.	Testbarkeit und verbleibende Grenzen	45
6.	Qualitätssicherung und Evaluation	46
6.1.	Evaluationsfragen und Analyseeinheit	46
6.2.	Untersuchungsstand und Artefakte	47
6.3.	Prüfprotokoll und Nachverfolgbarkeit	49
6.4.	Bewertungslogik	51
6.4.1.	Prüfung von Sicherheits- und Integritätsinvarianten	51
6.4.2.	Trennung technischer und semantischer Evaluation	53
6.5.	Befunde entlang der Prozessgrenzen	54
6.5.1.	Ereignisannahme und Zugriffsschutz	54
6.5.2.	Modellgrenze und Ausgabequalität	54
6.5.3.	Persistenz und Publish	55
6.6.	Überleitung zu den Ergebnissen	55
7.	Ergebnisse	56
7.1.	Konsolidierter Ergebnisstand	56
7.2.	Illustrierter Ablauf	58
7.3.	Zustandslogik des Vorschlags	58
7.4.	Reichweite des Ergebnisses	60
8.	Diskussion	61
8.1.	Interpretation des Ergebnisbeitrags	61
8.1.1.	Formale Struktur und semantische Qualität	61
8.1.2.	Rückfragen und Unsicherheitssteuerung	62
8.2.	Nutzenhypothesen, Aufwand und Alternativen	62

8.3. Architektonische Zielkonflikte	63
8.4. Organisatorische Einbettung menschlicher Kontrolle	64
8.5. Gültigkeit der Untersuchung	65
8.6. Risiken und verantwortbarer Einsatz	66
8.7. Übertragbarkeit	66
8.8. Diskussionsfazit	67
9. Fazit und Ausblick	68
9.1. Beantwortung der Forschungsfragen	68
9.1.1. Erforderliche Informationen eines Tickets	68
9.1.2. Kontrollierte Integration eines KI-Assistenten	69
9.2. Wissenschaftlicher und praktischer Beitrag	70
9.3. Priorisierter Ausblick	70
9.4. Schlussfolgerung	71
Quellen	72
A. Illustrierter Ticketverlauf	77
B. Dokumentierter Prüfstand	79
C. Ausgabeschema des Prototyps	80

Abbildungsverzeichnis

4.1. Systemkontext und Komponenten des Dev-Assist-Prototyps mit Vertrauensgrenzen. Der Express-Dienst orchestriert GitLab-Zugriffe, KI-Analyse und lokale Kontextpersistenz. Rote Datenflüsse überschreiten eine Plattform-, Prozess- oder Anbietergrenze. Eigene Darstellung auf Grundlage des untersuchten Quellcodes, Stand 31.07.2026.	28
4.2. Sequenz der Analyse- und Publish-Verarbeitung über den GitLab-Webhook. Die Route bestätigt angenommene Ereignisse früh mit HTTP 202 und führt Process beziehungsweise Publish anschließend ohne Rückkanal zum Aufrufer aus. Rote Hinweise kennzeichnen implementierte Fehlerpfade und die fehlende Kopplung zwischen sichtbarer Vorschau, gespeichertem Kontext und Freigabe. Eigene Darstellung auf Grundlage des untersuchten Quellcodes, Stand 31.07.2026.	33
7.1. Soll-Zustandsmodell eines Dev-Assist-Vorschlags und Abweichung der aktuellen Implementierung. Die Freigabe muss an dieselbe Revision gebunden sein, die vollständig in GitLab sichtbar war. Der rot gestrichelte Übergang zeigt die fehlende Status-, Revisions- und Freigabeprüfung. Eigene Darstellung auf Grundlage des untersuchten Quellcodes, Stand 31.07.2026.	59

Tabellenverzeichnis

2.1. Einordnung aktueller empirischer Arbeiten im Verhältnis zu Dev-Assist	12
3.1. Herkunft und Nachverfolgbarkeit der Anforderungsgruppen	20
3.2. Informationsgruppen des abgeleiteten Qualitätsmodells	22
4.1. Architekturbereiche, Verantwortlichkeiten und zentrale Module	27
4.2. Fehlerklassen und vorgesehene Reaktionen im Zielablauf	34
5.1. Leistungsstand der Kernfunktionen im untersuchten Prototyp	36
5.2. Projektbereiche und umgesetzte Verantwortlichkeiten	38
6.1. Ergebnisse des dokumentierten Build- und Testlaufs	48
6.2. Untersuchte projektinterne Artefaktgruppen	48
6.3. Prüfprotokoll und zulässige Schlussfolgerungen	50
7.1. Konsolidierter Anforderungs- und Nachweisstand des untersuchten Pro- totyps (eigene Darstellung)	57
8.1. Wirtschaftliche Einordnung der Aufbereitungsalternativen ohne Wir- kungsnachweis	63
8.2. Zentrale Bedrohungen der Gültigkeit und ihre Konsequenzen	65
9.1. Priorisierte Roadmap vom Proof of Concept zur belastbaren Evaluation	71
B.1. Kurzprotokoll des dokumentierten Build- und Testlaufs	79
C.1. Felder des kompakten Ausgabeschemas	80

Abkürzungsverzeichnis

API	Application Programming Interface
CLI	Command Line Interface
FA	funktionale Anforderung
HMAC	Hash-based Message Authentication Code
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
JSONL	JSON Lines
LLM	Large Language Model
NFA	nicht-funktionale Anforderung
NIST	National Institute of Standards and Technology
OCR	Optical Character Recognition
OWASP	Open Worldwide Application Security Project
REST	Representational State Transfer
SRS	Software Requirements Specification
TTL	Time to Live

1. Einleitung

Software-Tickets sind in agilen Entwicklungsprozessen zugleich Anforderung, Kommunikationsraum und Übergabepunkt. Ihre Qualität bestimmt, ob eine Aufgabe verstanden, umgesetzt und überprüft werden kann. Forschung zu Bug Reports zeigt jedoch eine wiederkehrende Lücke zwischen den Informationen, die meldende Personen bereitstellen, und den Angaben, die Entwicklungsteams benötigen. Besonders Reproduktionsschritte, Testfälle, Systemausgaben und technische Kontextinformationen fehlen häufig oder entstehen erst im weiteren Kommentarverlauf (vgl. Bettenburg et al., 2008, S. 308 f.; Breu et al., 2010, S. 301 ff.). Auch kurze User Stories können syntaktisch korrekt, aber semantisch unvollständig oder praktisch nicht nutzbar sein (vgl. Lucassen et al., 2016, S. 383 f. und S. 386 f.). Ein Ticket ist daher nicht allein wegen seiner Form oder Länge entwicklungsfähig.

In GitLab verteilt sich der relevante Kontext häufig auf Titel, Beschreibung, Kommentare, Anhänge und spätere Entscheidungen. Vor Beginn der Implementierung müssen diese Informationen manuell zusammengeführt, Widersprüche geklärt und Akzeptanzkriterien formuliert werden. Generative Sprachmodelle können natürlichsprachliche Inhalte flexibel klassifizieren, zusammenfassen und umstrukturieren. Ihre Ausgaben sind jedoch probabilistisch und können überzeugend formulierte, aber unbelegte Aussagen enthalten. NIST bezeichnet dieses Risiko als Confabulation beziehungsweise Hallucination (vgl. National Institute of Standards and Technology, 2024, S. 4 und S. 6). Für einen produktnahen Entwicklungsworkflow genügt deshalb weder eine reine Textgenerierung noch ein guter Prompt. Erforderlich ist eine kontrollierte Anwendung, die Kontext auswählt, Antworten strukturell prüft und Änderungen erst nach menschlicher Freigabe übernimmt.

Gegenstand dieser Arbeit ist der Prototyp Dev-Assist. Der TypeScript-/Express-Dienst reagiert auf adressierte GitLab-Ereignisse, sammelt Issue-, Kommentar- und optionalen Repository-Kontext und übergibt ihn über OpenCode an einen Analyseagenten. Je nach Informationslage veröffentlicht die Anwendung Rückfragen oder einen strukturierten Ticketvorschlag. Im regulären Webhook-Workflow übernimmt erst `@dev-assist publish` den lokal gespeicherten Stand; daneben existieren manuelle HTTP- und CLI-Wege. Das Modell besitzt keine unmittelbare Schreibberechtigung für GitLab.

Die Untersuchung folgt zwei Forschungsfragen:

1. **Forschungsfrage 1:** Welche Informationen muss ein GitLab-Ticket enthalten, damit Entwicklerinnen und Entwickler es nachvollziehbar umsetzen und überprüfen können?
2. **Forschungsfrage 2:** Wie lässt sich ein KI-gestützter Developer Assistant so in einen GitLab-basierten Entwicklungsworkflow integrieren, dass Analyse, Rückfragen, Vorschlag und Freigabe kontrolliert ablaufen?

Ziel ist ein technischer Machbarkeitsnachweis, nicht der Ersatz menschlicher Anforderungsarbeit. Methodisch werden zunächst Qualitätskriterien aus Requirements-Engineering- und Bug-Report-Literatur abgeleitet. Daraus folgen Anforderungen und eine Systemarchitektur, die anschließend als Prototyp umgesetzt werden. Die Evaluation verbindet Build, automatisierte Tests, Anforderungsabgleich und eine explizite Betrachtung der Aussagegrenzen. Eine Feldstudie oder ein belastbarer Wirksamkeitsnachweis realer Modellantworten ist nicht Bestandteil der Arbeit.

Das Erkenntnisinteresse liegt damit auf zwei Ebenen. Die fachliche Ebene untersucht, welche Informationsarten ein Ticket zu einem nutzbaren Arbeitsartefakt machen. Die technische Ebene untersucht, welche deterministischen Grenzen erforderlich sind, wenn eine probabilistische Modellkomponente in einen bestehenden Workflow eingebunden wird. Diese Ebenen werden nicht unabhängig betrachtet: Das fachliche Qualitätsmodell bestimmt das Ausgabeschema und die Rückfragen; Risiken und technische Grenzen bestimmen wiederum, welche Aussagen ein Vorschlag enthalten darf und wie er freigegeben wird.

Die Arbeit verwendet einen konstruktiven, verifikationsorientierten Aufbau. Literatur und externe Dokumentation begründen Qualitätskriterien sowie Schnittstellenverträge. Projektbeschreibung und Ist-Prozess liefern den Anwendungskontext. Architektur und Implementierung bilden das konstruierte Artefakt. Dessen technische Eigenschaften werden mit automatisierten Tests untersucht und anschließend im Hinblick auf die Forschungsfragen interpretiert. Aussagen über praktische Wirksamkeit werden bewusst von Aussagen über technische Funktionsfähigkeit getrennt.

1.1. Forschungsdesign

Das Forschungsdesign folgt der Ableitungskette Problem, Qualitätsmodell, Anforderungen, Architektur, Implementierung und Evaluation. Ausgangspunkt ist die in der Literatur beschriebene Informationslücke in Software-Tickets, nicht der vorhandene Quellcode. Die Literatursauswahl ist keine vollständige systematische Studie, sondern konzentriert sich auf unmittelbar relevante Qualitätsmerkmale von User Stories und Bug Reports, automatisierte Qualitätshilfen und Risiken generativer Systeme. Die Kategorien werden auf den GitLab-Kontext übertragen; dadurch bleiben Forschungsbefund, externer Schnittstellenvertrag und projektspezifische Festlegung unterscheidbar.

Untersuchungseinheit ist der durch ein GitLab-Ereignis ausgelöste Analyse- beziehungsweise Publish-Vorgang mit seinen Eingaben, Komponentenübergängen und Ausgaben. Diese Perspektive ist weiter als die Bewertung eines einzelnen Modelltexts, weil auch Aktivierung, Kontextauswahl, Validierung und Freigabe zur Forschungsfrage gehören. Sie ist zugleich enger als der vollständige Softwarelebenszyklus: Ticketqualität bezeichnet hier die Entwicklungsfähigkeit eines Arbeitsartefakts, nicht Produktqualität, Teamleistung oder fachliche Priorität.

Die Evidenz wird entsprechend getrennt. Literatur begründet das fachliche Soll, externe Dokumentation die Schnittstellen und der Repository-Stand das technische Ist. Der dokumentierte Build- und Testlauf belegt konkrete Eigenschaften deterministischer Komponenten; der Soll-Ist-Abgleich weist Erfüllung und Abweichungen aus. Daraus folgt weder automatisch semantisch richtiger Modelloutput noch ein sicherer Produktivbetrieb.

Diese Zuordnung erhöht die Nachvollziehbarkeit der Schlussfolgerungen: Jede Bewertung muss erkennen lassen, ob sie sich auf einen Literaturbefund, einen dokumentierten Vertrag, beobachtbaren Code oder das Ergebnis des dokumentierten Testlaufs stützt.

1.2. Abgrenzung und Aufbau

Nutzerexperimente, Langzeitdaten und ein statistisch ausgewerteter Ticketkorpus sind nicht Bestandteil der Arbeit. Ebenso werden Anhänge und Screenshots nicht seman-

tisch analysiert. Die Untersuchung kann daher technische Machbarkeit und konkrete Gestaltungsgrenzen zeigen, aber keine allgemeine Zeitersparnis oder Qualitätssteigerung im Feld nachweisen. Diese Abgrenzung bestimmt die Interpretation der Evaluation und der abschließenden Forschungsfragen.

Kapitel 2 legt die fachlichen und technischen Grundlagen. Kapitel 3 überführt sie in Anforderungen, Kapitel 4 in die Architektur und Kapitel 5 in die Implementierung. Kapitel 6 beschreibt die Evaluation. Darauf aufbauend stellen Kapitel 7 und 8 Ergebnisse und Diskussion getrennt dar; Kapitel 9 beantwortet die Forschungsfragen abschließend und priorisiert weiteren Forschungs- und Entwicklungsbedarf. Ein illustrierter Ticketverlauf, der dokumentierte Prüfstand und das Ausgabeschema sind im Anhang zusammengestellt.

2. Theoretische und technische Grundlagen

Dieses Kapitel bündelt die Grundlagen, die unmittelbar zur Gestaltung von Dev-Assist beitragen. Zunächst wird Ticketqualität aus dem Requirements Engineering und aus Forschung zu Bug Reports und User Stories hergeleitet. Anschließend werden Sprachmodelle und agentenbasierte Werkzeugnutzung eingeordnet. Den Abschluss bilden GitLab als Integrationsplattform sowie die daraus entstehenden Sicherheits- und Validierungsanforderungen.

2.1. Requirements Engineering und Ticketqualität

Requirements Engineering umfasst das Erheben, Analysieren, Dokumentieren, Prüfen und Verwalten von Anforderungen über ihren Lebenszyklus. Die öffentlich zugängliche Katalogseite zu ISO/IEC/IEEE 29148 beschreibt Requirements Engineering als Prozessbereich, der Anforderungen und zugehörige Informationsprodukte über den Lebenszyklus hervorbringt (vgl. ISO/IEC/IEEE, 2018, o. S.). Detaillierte normative Abschnittsvorgaben des Volltextes werden nicht vorausgesetzt. Qualitätsmerkmale wie Eindeutigkeit, Vollständigkeit, Nachverfolgbarkeit und Prüfbarkeit werden in der Fachliteratur mit diesem Standard verbunden und dort empirisch operationalisiert (vgl. Lubos et al., 2024, S. 389–397; Paiva et al., 2026, Abschnitte 3 und 5). Die öffentlich zugängliche Katalogseite zu ISO/IEC 25010 ordnet Qualitätsmodelle als Hilfsmittel ein, um Qualitätsanforderungen zu spezifizieren, Testziele abzuleiten und Akzeptanzkriterien zu bestimmen (vgl. ISO/IEC, 2023, o. S.). Die Katalogseiten dienen als begrifflicher Bezugsrahmen; die konkrete Operationalisierung für GitLab-Tickets wird mit empirischer Fachliteratur begründet.

2.1.1. Normativer Bezugsrahmen und eigene Operationalisierung

Die beiden Bezugspunkte erfüllen unterschiedliche Funktionen. ISO/IEC/IEEE 29148 wird in der Fachdiskussion auf den Requirements-Engineering-Prozess sowie auf Eigenschaften einzelner Anforderungen bezogen. Für diese Arbeit sind insbesondere eindeutige Formulierung, notwendige Information, Prüfbarkeit und Nachverfolgbarkeit relevant. Daraus folgt kein Anspruch, ein GitLab-Ticket sei eine vollständige normkonforme

Spezifikation. Das Ticket ist ein kompaktes, fortgeschriebenes Arbeitsartefakt. Die mit dem Standard verbundenen Kriterien helfen, Aussagen und Lücken zu beurteilen; die Auswahl der Ticketfelder ist eine begründete Operationalisierung dieser Arbeit.

ISO/IEC 25010 wird dagegen auf Qualitätseigenschaften eines Systems beziehungsweise Softwareprodukts bezogen. Merkmale wie funktionale Eignung, Zuverlässigkeit, Sicherheit, Wartbarkeit und Interaktionsfähigkeit können Qualitätsanforderungen oder spätere Prüfziele strukturieren. Sie bewerten nicht unmittelbar, ob ein Ticket sprachlich eindeutig formuliert ist. Für Dev-Assist wird dieser Rahmen auf der Systemebene genutzt: Authentifizierung und minimale Berechtigungen betreffen Sicherheit, fehlersichere Verarbeitung die Zuverlässigkeit, modulare Schnittstellen die Wartbarkeit und die Einbettung in GitLab die Interaktionsfähigkeit. Eine Aussage über Zeitersparnis oder Akzeptanz im Nutzungskontext folgt daraus nicht und benötigt eigene empirische Daten.

Die Übertragung auf das Qualitätsmodell erfolgt über beobachtbare Indikatoren. Eindeutigkeit wird nicht durch eine automatische Stilbewertung behauptet, sondern durch einen klar bezeichneten Gegenstand, Scope und sichtbare Widersprüche unterstützt. Prüfbarkeit wird über beobachtbare Akzeptanzkriterien operationalisiert. Nachverfolgbarkeit verlangt, belegte Ticket-, Kommentar- und Repository-Informationen von Modellannahmen und offenen Punkten zu unterscheiden. Vollständigkeit wird ticketartsspezifisch beurteilt: Bei einem Bug Report können Reproduktionsschritte wesentlich sein, bei einer Funktion Nutzen und fachliche Abgrenzung. Kein einzelnes ausgefülltes Feld beweist daher für sich die Erfüllung eines Normmerkmals.

Diese Trennung begrenzt auch die spätere Evaluation. Schema-Validierung kann das Vorhandensein und den Datentyp eines Feldes prüfen; sie kann nicht feststellen, ob dessen Inhalt eindeutig, wahr oder für den konkreten Stakeholder ausreichend ist. Die Arbeit beansprucht weder eine Zertifizierung noch eine Konformitätsprüfung gegen die Volltexte der Normen. Sie nutzt die öffentlichen Katalogseiten zusammen mit der Fachliteratur als begrifflichen Rahmen und legt offen, an welchen Stellen daraus projektspezifische Prüfregele abgeleitet werden.

2.1.2. Empirische Befunde zu User Stories und Bug Reports

In agilen Prozessen entstehen Anforderungen häufig in knappen Artefakten wie User Stories oder Tickets. Das erleichtert iterative Zusammenarbeit, erhöht aber die Abhän-

gigkeit vom gemeinsamen Kontext. Inayat et al. beschreiben agiles Requirements Engineering deshalb als stark kommunikations- und abstimmungsorientiert (vgl. Inayat et al., 2015, S. 915 ff.). Lucassen et al. unterscheiden bei User Stories syntaktische, semantische und pragmatische Qualität (vgl. Lucassen et al., 2016, S. 386 f.). Ein Text kann demnach formal lesbar sein und dennoch die falsche Bedeutung vermitteln oder für die Umsetzung unbrauchbar bleiben.

Bug Reports konkretisieren, welche Informationen in technischen Tickets fehlen können. Bettenburg et al. zeigen, dass Entwicklungsteams Reproduktionsschritte, Stack Traces und Testfälle besonders hilfreich finden, meldende Personen diese Angaben jedoch nicht immer bereitstellen können (vgl. Bettenburg et al., 2008, S. 308 f.). Breu et al. untersuchten Fragen in 600 Bug Reports und identifizierten unter anderem Nachfragen zu Reproduktion, Version, Betriebssystem, Beispielen, Ausgaben und Screenshots (vgl. Breu et al., 2010, S. 301 und S. 303). Chaparro et al. heben beobachtetes Verhalten, erwartetes Verhalten und Reproduktionsschritte als zentrale, zugleich häufig fehlende Bestandteile hervor (vgl. Chaparro et al., 2017, S. 396 f.).

Diese Befunde lassen sich nicht unverändert auf jede Ticketart übertragen. Sie begründen jedoch einen gemeinsamen Qualitätskern: Ein Ticket benötigt einen eindeutigen Gegenstand, Ziel und Kontext, erwartetes beziehungsweise gewünschtes Verhalten, eine Abgrenzung, überprüfbare Kriterien sowie sichtbare offene Punkte. Bei Fehlern kommen Ist-Verhalten, Soll-Verhalten und Reproduktionsschritte hinzu; bei neuen Funktionen stehen Nutzen, Scope und Akzeptanzbedingungen im Vordergrund. Technische Evidenz ist relevant, darf eine fehlende fachliche Entscheidung aber nicht durch Annahmen ersetzen.

GitLab-Issues sind dabei keine statischen Texte. Die Beschreibung bildet nur den Ausgangspunkt; Rückfragen und Entscheidungen entstehen häufig in Notes und Discussions. Ein Assistenzsystem muss deshalb den Gesprächsverlauf berücksichtigen und geklärte Informationen in das Ticket zurückführen. Qualität bedeutet in diesem Kontext nicht maximalen Umfang, sondern die nachvollziehbare Trennung relevanter Aussagen, Evidenz und Unsicherheit.

2.2. Automatisierte Qualitätsunterstützung

Die Forschung zeigt verschiedene Möglichkeiten, Ticketqualität teilweise automatisiert zu unterstützen. CUEZILLA bewertet Bug Reports und gibt Hinweise auf ergänzende Informationen. Der Ansatz greift damit die von Bettenburg et al. identifizierte Lücke zwischen bereitgestellten und benötigten Angaben auf (vgl. Bettenburg et al., 2008, S. 308 f.). Für User Stories operationalisiert AQUA Kriterien des Quality User Story Frameworks und macht formale sowie sprachliche Defekte sichtbar (vgl. Lucassen et al., 2016, S. 383 f. und S. 386 f.). Beide Ansätze verdeutlichen, dass Qualitätsmerkmale in Regeln oder Prüfverfahren übersetzt werden können.

Chaparro et al. entwickelten DeMIBuD zur Erkennung fehlenden erwarteten Verhaltens und fehlender Reproduktionsschritte in Bug Descriptions (vgl. Chaparro et al., 2017, S. 397 und S. 401 ff.). Song und Chaparro stellen mit BEE ein Werkzeug vor, das Bug Reports klassifiziert, die Bestandteile Observed Behavior, Expected Behavior und Steps to Reproduce markiert und fehlende Elemente meldet (vgl. Song/Chaparro, 2020, S. 1551 f.). Diese Systeme arbeiten auf klar abgegrenzten Qualitätsaspekten und zeigen, dass strukturierende Rückmeldung technisch möglich ist.

Mit ChatBR übertragen Bo et al. diese Idee auf einen hybriden Ansatz: Ein spezialisierter Klassifikator erkennt beobachtetes Verhalten, erwartetes Verhalten und Reproduktionsschritte; ein Sprachmodell ergänzt als fehlend klassifizierte Bestandteile. Die Autoren berichten über sechs Projekte eine hohe strukturelle Vollständigkeit und eine mittlere semantische Ähnlichkeit der Ergänzungen von 84,10 Prozent (vgl. Bo et al., 2024, S. 1472–1483). Die Messung zeigt, dass generative Ergänzungen strukturell anschlussfähig sein können. Sie belegt jedoch nicht, dass eine Ergänzung im konkreten Projekt faktisch richtig ist. Für Dev-Assist folgt daraus die Trennung von Defekterkennung, Generierung und menschlicher Freigabe.

Dev-Assist übernimmt nicht unverändert eines dieser Verfahren. Der Prototyp richtet sich an unterschiedliche GitLab-Ticketarten und berücksichtigt zusätzlich Kommentare sowie begrenzten Repository-Kontext. Statt ausschließlich Defekte zu markieren, soll er bei fehlender Grundlage Rückfragen und bei ausreichendem Kontext einen konsolidierten Entwurf erzeugen. Der wissenschaftliche Anschluss liegt in den Qualitätskategorien; der eigene Beitrag liegt in ihrer Verbindung mit einem ereignisbasierten, menschlich kontrollierten GitLab-Workflow.

Diese Erweiterung schafft zugleich neue Bewertungsprobleme. Ein regelbasiertes Werkzeug kann anhand definierter Muster geprüft werden. Ein generatives Modell kann sprachlich unterschiedliche, aber inhaltlich ähnliche Ergebnisse liefern und zugleich unbelegte Informationen ergänzen. Deshalb sind stabile Ausgabeform, Faktentreue, Relevanz der Rückfragen und Nutzbarkeit getrennt zu betrachten. Die technische Validierung des Datenformats ist notwendig, reicht als Wirksamkeitsnachweis aber nicht aus.

2.3. Aktuelle Forschung zu LLM-gestütztem Requirements Engineering

Die jüngere Forschung untersucht Sprachmodelle nicht mehr nur als allgemeine Textgeneratoren, sondern für konkrete Aufgaben des Requirements Engineering. Drei Richtungen sind für Dev-Assist besonders relevant: die Generierung oder Überarbeitung von Anforderungen, die Qualitätsprüfung gegen definierte Kriterien und die Verknüpfung verteilter Kontextquellen. Die Studien liefern erste empirische Evidenz, zeigen aber zugleich, dass formal überzeugende Texte und ein kontrollierter Arbeitsprozess getrennte Untersuchungsgegenstände bleiben.

Krishna et al. vergleichen GPT-4 und CodeLlama bei der Erstellung, Validierung und Korrektur von Software Requirements Specifications. Die erzeugten Dokumente wurden anonymisiert und von unabhängigen Personen anhand mehrerer Qualitätskriterien bewertet; zusätzlich betrachteten die Autoren vier Anwendungsfälle und den Bearbeitungsaufwand. Die Ergebnisse zeigen Potenzial für strukturierte Entwürfe und Zeiterparnis, unterscheiden sich jedoch nach Modell und Aufgabe. Insbesondere fiel die Validierungsleistung nicht für beide Modelle gleich aus (vgl. Krishna et al., 2024, S. 475–483). Für Dev-Assist folgt daraus weder eine übertragbare Qualitätsgarantie noch ein Produktivitätsnachweis. Die Studie arbeitet mit vollständigen Spezifikationsdokumenten und ausgewählten Use Cases, während hier kurze, dialogisch ergänzte Tickets in einem laufenden Workflow betrachtet werden.

Lubos et al. untersuchen ein LLM als Prüfinstanz für die in ISO/IEC/IEEE 29148 beschriebenen Qualitätsmerkmale einzelner Anforderungen. Teilnehmende bewerteten Anforderungen zunächst unabhängig und anschließend die Entscheidungen, Erläuterungen und Verbesserungsvorschläge des Modells. Die anfängliche Übereinstimmung zwischen Menschen und Modell war teilweise schwach; Erklärungen und überarbeite-

te Formulierungen wurden gleichwohl häufig als plausibel beziehungsweise verbessert eingeschätzt (vgl. Lubos et al., 2024, S. 389–397). Der Befund stützt einen assistierenden Einsatz, warnt aber vor der Gleichsetzung einer Modellbewertung mit fachlichem Konsens. Dev-Assist benötigt daher eine sichtbare Begründungs- und Freigabestufe, statt den Modelloutput als Qualitätsurteil zu behandeln.

Besonders nah am untersuchten Artefakt liegt die Studie von Paiva et al. Sie erzeugt aus Issue-Titeln von fünf aktiven Open-Source-Projekten Anforderungssätze und vergleicht zwei Modelle sowie drei Promptstrategien. Bewertet werden Unambiguität, Prüfbarkeit und Singularität nach ISO/IEC/IEEE 29148. Die Autoren begrenzen ihre Aussage ausdrücklich auf sprachlich wohlgeformte Anforderungen bei minimalem Eingangskontext; Projektangemessenheit, Vollständigkeit und Stakeholder-Abstimmung werden nicht nachgewiesen (vgl. Paiva et al., 2026, Abschnitte 3 und 5). Gerade diese Grenze begründet den breiteren Kontextansatz von Dev-Assist: Titel, Beschreibung, Kommentare und ausgewählte Repository-Hinweise werden gemeinsam betrachtet. Ob dieser zusätzliche Kontext tatsächlich bessere Vorschläge erzeugt, muss jedoch empirisch geprüft werden.

K C et al. untersuchen unmittelbar die Verfeinerung von Feature Requests. Auf Grundlage von 476 Requests aus zwei Open-Source-Projekten erkannten die Autoren Ambiguitäten und Unvollständigkeit und erzeugten dazu konkrete Klärungsfragen. In der manuellen Bewertung wurden 164 von 167 Fragen als korrekt beurteilt; erfahrene Entwicklerinnen und Entwickler bestätigten zugleich, dass Nachfragen bei mangelhaften Requests typischerweise der erste Bearbeitungsschritt sind (vgl. K C et al., 2025, S. 104–116). Semantische und pragmatische Mehrdeutigkeiten blieben schwieriger als lokale sprachliche Defekte. Der Befund stützt den dialogischen Ansatz von Dev-Assist, nicht aber eine automatische Freigabe: Eine passende Frage kann fehlendes Domänenwissen sichtbar machen, es jedoch nicht selbst ersetzen.

Industrielle Studien schärfen diese Grenze. Bashir et al. zeigen an drei proprietären Anforderungsdatensätzen, dass projektnah ausgewählte In-Context-Beispiele die Ambiguitätserkennung gegenüber Zero-Shot-Prompts im Mittel deutlich verbessern; Leistung und Erklärungsqualität bleiben dennoch von Datensatz, Prompt und Domäne abhängig (vgl. Bashir et al., 2025, S. 620–631). Mahbub et al. berichten bei der Analyse einer Spezifikation für ein mechanisches Beatmungsgerät eine Präzision von 0,61 für Unvollständigkeit, 0,43 für Inkonsistenz und 0,39 für Ambiguität. Als Grenzen nennen

sie insbesondere fehlende dokumentweite Querverweise und unzureichendes Spezialwissen (vgl. Mahbub et al., 2024, S. 171972–171992). Beide Studien sprechen dafür, Modellurteile als prüfbare Hinweise und nicht als abschließende Qualitätsentscheidung zu behandeln.

Eine mögliche Antwort auf fehlenden Projektkontext ist Retrieval-Augmented Generation. Ibtasham et al. evaluieren ReqRAG mit sieben technischen Industriedokumenten und 27 realen Fragen samt Referenzantworten. Rund 70 Prozent der Antworten wurden von Praktikerinnen und Praktikern als angemessen, nützlich und relevant bewertet; die Einzelurteile streuten jedoch deutlich (vgl. Ibtasham et al., 2025, S. 277–292). Der Abruf zusätzlichen Kontexts kann Antworten demnach fachlich besser fundieren, garantiert aber weder Vollständigkeit noch Richtigkeit. Für Dev-Assist begründet dies die Herkunftskennzeichnung und Begrenzung des Repository-Kontexts sowie eine gesonderte Prüfung unbelegter Ergänzungen.

Tizard et al. zeigen ergänzend, dass Anforderungen in Open-Source-Projekten ein Ökosystem aus Forenbeiträgen, Issue Tracker und Produktdokumentation bilden. In ihrer Untersuchung von VLC und Firefox ließen sich 275 Forenbeiträge automatisiert mit inhaltlich verwandten Issues abgleichen; zugleich blieben Fehlzuordnungen und manueller Prüfbedarf bestehen (vgl. Tizard et al., 2023, S. 2381 f. und S. 2389 ff.). Für diese Arbeit ist weniger das konkrete Matching-Verfahren als der empirische Nachweis verteilter Kontextquellen relevant. Er stützt die Entscheidung, Kommentare und Repository-Informationen nicht als optionales Beiwerk, sondern als potenzielle Evidenz mit eigener Herkunft zu behandeln.

Frattini et al. kritisieren, dass Requirements-Quality-Forschung häufig normative Formulierungsregeln bewertet, ohne deren Wirkung auf nachgelagerte Entwicklungsaktivitäten ausreichend zu untersuchen. Ihr harmonisiertes Modell trennt die Eigenschaft einer Anforderung, die Bewertung dieser Eigenschaft und die Wirkung im Nutzungskontext (vgl. Frattini et al., 2023, S. 507–520). Diese Unterscheidung ist für Dev-Assist zentral: Ein schema-konformer und sprachlich klarer Vorschlag ist zunächst nur ein formal verarbeitetes Artefakt. Ob er Implementierung, Prüfung oder Abstimmung verbessert, erfordert davon getrennte Nutzungsdaten.

Tabelle 2.1: Einordnung aktueller empirischer Arbeiten im Verhältnis zu Dev-Assist

Arbeit	Untersuchter Beitrag	Konsequenz für diese Arbeit
Krishna et al. (2024)	Generierung und Validierung vollständiger SRS-Dokumente mit zwei LLMs	Potenzial ist aufgaben- und modellabhängig; Ticketwirkung separat prüfen
Lubos et al. (2024)	LLM-basierte Qualitätsprüfung nach ISO/IEC/IEEE 29148 mit menschlicher Bewertung	Modellerklärung unterstützt Prüfung, ersetzt aber keinen fachlichen Konsens
K C et al. (2025)	Erkennung von Lücken und Generierung konkreter Klärungsfragen für Feature Requests	Dialog unterstützt Verfeinerung; semantische Lücken bleiben menschlich zu prüfen
Bo et al. (2024)	Hybride Erkennung und generative Ergänzung fehlender Bug-Report-Bestandteile	Strukturelle Vollständigkeit nicht mit faktischer Richtigkeit gleichsetzen
Tizard et al. (2023)	Verknüpfung von Forum, Issue Tracker und Dokumentation	Kontextquellen gemeinsam betrachten und Herkunft erhalten
Paiva et al. (2026)	Anforderungsgenerierung aus Issue-Titeln mit mehreren Prompts und Modellen	Sprachqualität bei Minimalkontext belegt keine Projektangemessenheit
Ibtasham et al. (2025)	Industrielle kontextgestützte Beantwortung von Requirements-Fragen	Abruf zusätzlichen Kontexts unterstützt, garantiert aber keine korrekte Antwort
Frattini et al. (2023)	Theorie und Forschungsrahmen für Requirements Quality	Formale Artefaktqualität von Wirkung im Entwicklungsprozess trennen

Tabelle 2.1 verdeutlicht damit eine Forschungslücke, an die der Prototyp anschließt: Vorhandene Arbeiten evaluieren einzelne Generierungs-, Prüf- oder Verknüpfungsaufgaben, während die kontrollierte Zustandsfolge von Ereignisannahme, Rückfrage, sichtbarer Vorschau, Persistenz und Freigabe seltener gemeinsam betrachtet wird. Dev-Assist liefert für diese Lücke einen Architektur- und Implementierungsbeitrag. Ohne kuratiertes Ticketkorpus, wiederholte Modellläufe und fachkundige Bewertung bleibt seine inhaltliche Wirkung jedoch offen.

2.4. Vergleich der Ansätze und Ableitung des Qualitätsmodells

Die Arbeiten betrachten Ticketqualität aus unterschiedlichen Perspektiven. Bettenburg et al. und Breu et al. beschreiben empirisch, welche Informationen in Bug Reports fehlen und welche Nachfragen daraus entstehen (vgl. Bettenburg et al., 2008, S. 308 f.; Breu et al., 2010, S. 301 ff.). Lucassen et al. zerlegen User-Story-Qualität in syntaktische, semantische und pragmatische Aspekte und operationalisieren Teile davon mit AQUA (vgl. Lucassen et al., 2016, S. 383 f. und S. 386 f.). Chaparro et al. sowie Song und Chaparro fokussieren beobachtetes und erwartetes Verhalten sowie Reproduktionsschritte in Fehlerberichten (vgl. Chaparro et al., 2017, S. 396 f.; Song/Chaparro, 2020, S. 1551 f.).

Damit unterscheiden sich Artefaktart und Unterstützungsziel. User-Story-Verfahren bewerten vor allem Form und Nutzbarkeit einer knappen Anforderung; Bug-Report-Verfahren benötigen zusätzlich situationsbezogene Evidenz. DeMIBuD oder BEE erkennen abgegrenzte fehlende Bestandteile, während Dev-Assist Informationen über Kommentare hinweg konsolidieren und Rückfragen erzeugen soll. Gemeinsam ist den Ansätzen, dass sprachliche Vollständigkeit keine sachliche Richtigkeit garantiert.

Aus dem Vergleich entsteht deshalb ein ticketartübergreifendes, aber nicht starres Qualitätsmodell. **Ziel und Gegenstand** identifizieren Änderung und Nutzen. **Kontext und Scope** erfassen Voraussetzungen, Abhängigkeiten und Nicht-Ziele. **Verhalten und Kriterien** verbinden erwartete Wirkung mit prüfbaren Bedingungen und ergänzen bei Fehlern Ist-Verhalten und Reproduktion. **Evidenz und Hinweise** trennen belegte technische Angaben von fachlichen Entscheidungen. **Unsicherheit** erhält Fragen, Widersprüche, Risiken und Annahmen sichtbar.

Dev-Assist prüft somit zunächst, welche Kategorien für die Ticketart relevant und hinreichend belegt sind. Wesentliche Lücken führen zu konkreten Rückfragen; bei belastbarem Kern entsteht ein Vorschlag mit weiterhin sichtbarer Unsicherheit. Entscheidend ist nicht, jeden Abschnitt formal zu füllen, sondern unbelegte Sicherheit zu vermeiden. Das Modell verbindet damit Literaturbefunde und spätere Anforderungen: Struktur, Faktentreue und praktische Prüfbarkeit werden getrennt beurteilt.

Diese Operationalisierung verhindert zugleich, dass Umfang als Qualitätsmaß dient. Ein kurzer, belegter Vorschlag kann geeigneter sein als ein ausführlicher Text mit Annahmen. Kapitel 3 übersetzt die Kategorien deshalb in nachverfolgbare fachliche und

technische Anforderungen.

2.5. Large Language Models und agentenbasierte Assistenz

Large Language Models werden auf großen Textmengen vortrainiert und können Aufgaben anhand natürlichsprachlicher Anweisungen und Beispiele bearbeiten. Brown et al. zeigen mit GPT-3, dass Aufgaben ohne aufgabenspezifische Gewichtsaktualisierung über Kontext und Prompt gelöst werden können (vgl. Brown et al., 2020, S. 1 und S. 3). Zhao et al. ordnen Prompt Engineering als gezielte Gestaltung solcher Eingaben ein (vgl. Zhao et al., 2023, S. 44). Für Dev-Assist ist dies relevant, weil Ticketart, Sprache und Informationslage stark variieren und sich nicht vollständig durch feste Einzelregeln abbilden lassen.

Ein LLM kann vorhandene Aussagen gruppieren, fehlende Kategorien markieren, Rückfragen formulieren und einen strukturierten Entwurf erzeugen. Diese Flexibilität ist jedoch nicht mit gesichertem Systemverständnis gleichzusetzen. Das Modell kennt weder automatisch die fachliche Absicht noch den vollständigen Projektkontext. Seine Aufgabe wird daher auf Assistenz begrenzt: Es liefert eine Analyse nach einem vorgegebenen Ausgabeschema; die Anwendung prüft und verarbeitet sie.

LLM-basierte Agenten erweitern die reine Antwortgenerierung um Wahrnehmung und Handlungen. Xi et al. beschreiben dafür die Komponenten Brain, Perception und Action (vgl. Xi et al., 2023, S. 1). ReAct verbindet sprachliche Zwischenschritte mit Aktionen und externem Kontext (vgl. Yao et al., 2023, S. 1 f.); Toolformer untersucht die Nutzung externer Werkzeuge über APIs (vgl. Schick et al., 2023, S. 1). Für diese Arbeit ist weniger das jeweilige Verfahren als das Architekturprinzip entscheidend: Ein Modell wird erst durch kontrollierte Datenquellen, Schnittstellen und Berechtigungen zu einem Bestandteil des Entwicklungsworkflows.

OpenCode dient als Agentenschnittstelle. Konfigurierbare Agents bündeln Aufgabenbeschreibung, Modell und erlaubte Werkzeuge (vgl. OpenCode Documentation, o. J.a, Abschnitt "Agents"). Die CLI führt einen Agenten mit festgelegtem Modell und Ausgabeformat aus, ohne dass die aufrufende Anwendung die Modellgewichte selbst hosten muss (vgl. OpenCode Documentation, o. J.b, o. S.). Dev-Assist definiert in `opencode.json` zwei Agents: `dev-assist-analyzer` für die Ticketanalyse und `repo-summary` für eine

begrenzte Repository-Zusammenfassung. Der Aufruf erfolgt über `opcode run`; die Anwendung übergibt den vorbereiteten Prompt und wertet die CLI-Ausgabe anschließend selbst aus.

Entscheidend ist die Berechtigungsgrenze. Der Analyseagent erhält keine direkten GitLab-Rechte und keine freie Werkzeugauswahl für produktive Änderungen. Issue-Zugriff, Persistenz und Publish bleiben deterministische Aufgaben der Express-Anwendung. OpenCode ist damit die Ausführungsschnittstelle zur probabilistischen Komponente, nicht der Prozessverantwortliche. Diese Trennung reduziert Kopplung, unterstützt einen Mock-Modus ohne Modellkosten und macht die Grenzen um das Modell testbar.

2.6. Bewertung generativer Ticketvorschläge

Die Bewertung einer generierten Anforderung muss mehrere Ebenen unterscheiden. **Strukturelle Qualität** fragt, ob die erwarteten Felder vorhanden und maschinenlesbar sind. **Semantische Qualität** betrifft Faktentreue, Widerspruchsfreiheit und Übereinstimmung mit dem Ticketkontext. **Pragmatische Qualität** fragt, ob Entwicklung und Qualitätssicherung den Vorschlag tatsächlich verwenden können. Diese Trennung knüpft an Lucassen et al. an und verhindert, dass ein valides JSON-Objekt bereits als gute Anforderung gilt (vgl. Lucassen et al., 2016, S. 386 f.).

Für Rückfragen sind Präzision, Relevanz und Beantwortbarkeit entscheidend. Eine allgemeine Frage wie "Welche weiteren Informationen gibt es?" benennt keine konkrete Lücke. Eine Frage nach erwartetem Verhalten in einem bestimmten Szenario kann dagegen unmittelbar zur Verbesserung des Tickets beitragen. Wiederholte oder bereits beantwortete Fragen vermindern die Nutzbarkeit. Das System muss daher nicht nur Fragen erzeugen, sondern den vorhandenen Dialog berücksichtigen.

Akzeptanzkriterien sind dann nützlich, wenn sie ein beobachtbares Ergebnis statt einer Implementierungsaktivität beschreiben. "Eine Funktion implementieren" ist kein Prüfkriterium; eine nachvollziehbare Ein-/Ausgabebeziehung oder ein konkreter Fehlerzustand ist überprüfbar. Technischer Kontext wiederum benötigt eine erkennbare Quellenbasis. Repository-Informationen dürfen Hinweise stützen, aber fehlende fachliche Entscheidungen nicht als technische Notwendigkeit ausgeben.

Schließlich besitzt die Prozesswirkung eine eigene Ebene. Ein fachlich guter Vorschlag kann ungeeignet sein, wenn er ungefragt veröffentlicht, nicht nachvollziehbar gespeichert oder nur mit unverhältnismäßigem Prüfaufwand nutzbar ist. Umgekehrt kann ein kontrollierter Workflow keine inhaltlich schwache Modellantwort heilen. Eine belastbare Evaluation muss daher Systemverhalten und Antwortqualität getrennt messen und anschließend gemeinsam interpretieren.

2.7. GitLab und technische Integrationsplattform

GitLab bildet Issues, Kommentare, Status und Projektkontext an einem gemeinsamen Arbeitsort ab. Webhooks informieren externe Anwendungen per HTTP über Ereignisse; für Dev-Assist sind insbesondere Issue- und Comment-Events relevant (vgl. GitLab Documentation, o. J.a, Abschnitt "Webhook events"; GitLab Documentation, o. J.b, Abschnitt "Comment events"). Die Issues API stellt Titel und Beschreibung bereit und erlaubt deren Aktualisierung. Über die Notes API kann der Kommentarverlauf gelesen und ergänzt werden (vgl. GitLab Documentation, o. J.c, o. S.; GitLab Documentation, o. J.d, o. S.).

Der Prototyp verwendet Node.js, TypeScript und Express. Node.js stellt die serverseitige JavaScript-Laufzeit und HTTP-Grundlagen bereit (vgl. Node.js Documentation, o. J.a, o. S.; Node.js Documentation, o. J.b, o. S.). Express ergänzt Routing und Middleware für den Webhook-Endpunkt (vgl. Express Documentation, o. J., o. S.). TypeScript unterstützt die statische Prüfung interner Datenformen, kann externe Payloads und Modellantworten aber nicht zur Laufzeit garantieren (vgl. TypeScript Documentation, o. J., o. S.). Deshalb benötigt die Anwendung zusätzlich explizites Parsing und Laufzeitvalidierung.

Der technische Grundablauf lautet: GitLab sendet ein Ereignis an Express; die Anwendung prüft Authentizität und Relevanz, lädt gegebenenfalls aktuelles Issue und Notes, erstellt einen begrenzten Analysekontext und ruft OpenCode auf. Eine gültige Antwort wird als Rückfrage oder Vorschlag gerendert und persistiert. Erst eine neue, explizite Benutzerhandlung löst die Aktualisierung des Issues aus.

2.8. Risiken und Gestaltungsfolgen

Das zentrale inhaltliche Risiko sind falsche oder unbelegte Modellinhalte. Huang et al. beschreiben Halluzinationen als plausibel wirkende, aber nicht faktische Aussagen (vgl. Huang et al., 2023, S. 1 f.). Eine Schema-Prüfung kann falsche Feldtypen abweisen, nicht aber die Wahrheit eines syntaktisch gültigen Satzes feststellen. Dev-Assist muss Unsicherheit deshalb sichtbar erhalten, bei wesentlichen Lücken Rückfragen stellen und vor Publish eine menschliche Prüfung verlangen.

Ein zweites Risiko ist Prompt Injection. Direkte Tickettexte und indirekt eingebundene Inhalte können versuchen, Systemanweisungen zu überschreiben (vgl. Liu et al., 2023, S. 1; OWASP, 2025a, Abschnitt “LLM01:2025 Prompt Injection”). Ticket- und Repository-Inhalte sind daher als untrusted data zu behandeln. Der Agent erhält keine freie Werkzeugauswahl und keine produktive Schreibberechtigung. Diese Begrenzung adressiert zugleich das von OWASP beschriebene Risiko übermäßiger Handlungsfähigkeit (vgl. OWASP, 2025b, Abschnitt “LLM06:2025 Excessive Agency”).

Auch der Webhook-Eingang ist eine Sicherheitsgrenze. Beim aktuellen Signaturverfahren von GitLab sendet GitLab `webhook-id`, `webhook-timestamp` und `webhook-signature`. Signiert wird `{message_id}.{timestamp}.{body}`; das Ergebnis wird als Base64-Wert mit `v1,-`-Präfix übertragen. Eine Prüfung des Zeitstempels begrenzt Replay-Möglichkeiten (vgl. GitLab Documentation, o. J.a, Abschnitt “Signing tokens”). Zusätzlich müssen Bot-Schleifen, doppelte Lieferungen, ungültige IDs und Fehler externer Dienste berücksichtigt werden.

Aus den Grundlagen folgen vier Leitprinzipien für Dev-Assist: Erstens wird nur explizit adressierter Kontext verarbeitet. Zweitens sammelt die Anwendung Daten kontrolliert und trennt sie von festen Systemanweisungen. Drittens akzeptiert sie nur eine definierte, maschinenlesbare Antwort. Viertens erfolgen produktive Änderungen nach einer sichtbaren Vorschau und einem gesonderten Publish-Kommando. Kapitel 3 konkretisiert diese Prinzipien als Anforderungen.

2.9. Zwischenfazit

Die Grundlagen beantworten die Forschungsfragen noch nicht, grenzen aber ihren Lösungsraum ein. Für die erste Forschungsfrage zeigen Requirements Engineering, User-Story- und Bug-Report-Forschung, dass Ticketqualität mehrere unterscheidbare Informationsarten umfasst. Ziel, Kontext, erwartetes Verhalten, Prüfkriterien und Unsicherheit bilden einen gemeinsamen Kern; ticketartspezifische Evidenz ergänzt ihn. Kommentare sind Teil dieses Artefakts, weil fehlende Informationen häufig erst im Dialog entstehen.

Für die zweite Forschungsfrage zeigen Webhooks, APIs und agentenbasierte Ansätze, wie ein Modell technisch in GitLab eingebunden werden kann. Die Literatur zu Halluzinationen, Prompt Injection und übermäßiger Handlungsfähigkeit macht zugleich deutlich, dass Modellflexibilität keine hinreichende Architektur ist. Kontrolle muss außerhalb des Modells durch Authentifizierung, Kontextbegrenzung, minimale Berechtigungen, Laufzeitvalidierung und explizite Freigabe entstehen.

Damit ergibt sich eine Gestaltungsannahme: Ein KI-gestützter Assistant kann die Ticketaufbereitung unterstützen, wenn das fachliche Qualitätsmodell die erwartete Informationsstruktur vorgibt und eine deterministische Anwendungshülle alle Übergänge zum Modell und zu GitLab kontrolliert. Kapitel 3 übersetzt diese Annahme in konkrete Anforderungen. Die späteren Kapitel prüfen die technischen Voraussetzungen; eine Unterstützungswirkung kann ohne Ticketkorpus und Nutzungsevaluation nicht nachgewiesen werden.

3. Anforderungsanalyse

Die Anforderungsanalyse verbindet Literatur, bestehenden GitLab-Prozess, projektinterne Zielbeschreibung und technische Rahmenbedingungen. Sie definiert, welche Ticketqualität Dev-Assist unterstützen soll und welche Grenzen für den KI-gestützten Workflow gelten. Die Anforderungen dienen zugleich als Bezugspunkt für Architektur, Implementierung und Evaluation.

3.1. Vorgehen und Nachverfolgbarkeit

Die Anforderungen werden aus vier Quellen abgeleitet. Erstens liefern die in Kapitel 2 ausgewerteten Arbeiten Qualitätskategorien für User Stories und Bug Reports. Zweitens zeigt der manuelle Ist-Prozess, an welchen Stellen Kontext verteilt ist und Rückfragen entstehen. Drittens beschreibt die projektinterne Zielsetzung Aktivierung, Analyse, Klärung und Publish. Viertens begrenzen GitLab-Verträge, Sicherheitsrisiken und der gewählte Technologie-Stack die technische Ausgestaltung.

Die Ableitung folgt einer einfachen Nachverfolgbarkeitslogik. Literaturbefunde zu fehlenden Angaben führen zum Qualitätsmodell und zu FA-5 bis FA-7. Der dialogische Charakter von Tickets begründet FA-4 und FA-6. Das Ziel einer kontrollierten Prozessintegration führt zu FA-1 bis FA-3 sowie FA-9 bis FA-11. Risiken aus Webhooks, Prompt Injection und Halluzinationen begründen insbesondere NFA-3, NFA-5 bis NFA-8. Wartbarkeit und reproduzierbare Evaluation führen zu NFA-1, NFA-2, NFA-4 und NFA-9.

Die in Tabelle 3.1 zusammengefasste Zuordnung verhindert, dass Anforderungen nur aus dem vorhandenen Code rückwärts abgeleitet werden. Der Prototyp wird später gegen das fachliche Zielbild geprüft; Abweichungen bleiben sichtbar. Gleichzeitig werden Anforderungen auf einen Proof of Concept begrenzt. Produktivbetriebliche Funktionen werden nur aufgenommen, wenn sie eine zentrale Sicherheits- oder Prozessgrenze darstellen.

Tabelle 3.1: Herkunft und Nachverfolgbarkeit der Anforderungsgruppen

Quelle	Abgeleiteter Bedarf	Anforderungsbezug
Forschung zur Ticketqualität	Fehlende Angaben erkennen, Verhalten und Prüfbarkeit trennen	Qualitätsmodell, FA-5 bis FA-8
GitLab-Ist-Prozess	Beschreibung und Kommentare gemeinsam behandeln, Interaktion im Issue halten	FA-1 bis FA-4, FA-6, FA-9
KI- und Sicherheitsrisiken	Modelloutput begrenzen, Unsicherheit erhalten, menschlich freigeben	FA-8, FA-11, NFA-3, NFA-5 bis NFA-7
Betrieb und Evaluation	Module isolieren, Ereignisse nachvollziehen und externen Zugriff konfigurieren	NFA-1, NFA-2, NFA-4, NFA-8 bis NFA-10

3.2. Ausgangs- und Zielprozess

Im Ausgangsprozess erstellt eine Person ein Issue mit einer Anforderung, einem Fehler oder einer technischen Aufgabe. Fehlende Informationen werden in Kommentaren nachgefragt; Antworten, Screenshots und Entscheidungen ergänzen den Kontext schrittweise. Vor der Umsetzung muss eine bearbeitende Person diese Fragmente konsolidieren. Als problemleitende Annahme der Arbeit kann diese Fragmentierung Wartezeiten, wiederholte Rückfragen und schwer nachvollziehbare Übergaben begünstigen; eine Prozessmessung dazu ist nicht Bestandteil der Untersuchung.

Dev-Assist soll diese Zusammenarbeit unterstützen, ohne sie zu ersetzen. Der Zielprozess umfasst fünf Schritte: Eine Person adressiert den Assistant zu Beginn der ersten inhaltlichen Zeile mit `@dev-assist`. Die Anwendung prüft das Ereignis und sammelt Issue- sowie Kommentar-Kontext. Bei wesentlichen Lücken veröffentlicht sie gezielte Rückfragen; andernfalls erstellt sie einen strukturierten Vorschlag. Der Vorschlag bleibt zunächst sichtbar und unverbindlich. Erst `@dev-assist publish` übernimmt den gespeicherten Stand in das Issue.

Die beteiligten Rollen haben unterschiedliche Interessen. Meldende Personen benötigen eine geringe Einstiegshürde und verständliche Rückfragen. Product Owner benötigen Ziel, Nutzen und Abgrenzung; Entwicklung und Qualitätssicherung benötigen umsetzbare Aussagen und prüfbare Kriterien. Betreiber benötigen sichere Konfiguration, Logging und kontrollierbares Fehlerverhalten. Der Assistent muss deshalb fachliche Verständlichkeit und technische Nachvollziehbarkeit verbinden.

3.3. Stakeholder, Priorisierung und Nachverfolgbarkeit

Die Stakeholder-Perspektiven stehen teilweise in Spannung. Meldende Personen benötigen eine geringe Einstiegshürde; Entwicklung und Qualitätssicherung brauchen genügend Kontext für Annahmen und Prüfkriterien. Product Owner betrachten Nutzen und Scope, Betreiber sichere Zugriffe und Fehlerdiagnose. Der Assistent darf diesen Ausgleich nicht durch erfundene Inhalte herstellen, sondern muss entscheidungsrelevante Lücken gezielt sichtbar machen.

Muss-Anforderungen sichern deshalb Kernprozess und Sicherheitsgrenzen: Aktivierung, Abruf des Ticketkontexts, strukturierte Analyse, Persistenz, expliziter Publish sowie die Begrenzung externer Eingänge und Modellantworten. Soll-Anforderungen wie erweiterte Screenshot-Verarbeitung, zusätzliche GitLab-Metadaten und ausführlichere Diagnosen erhöhen den Nutzen. Dashboard, Produktmetriken und ein zusätzliches Rollenmodell bleiben Kann-Erweiterungen. Diese Priorisierung beschreibt das Soll und behauptet nicht, dass der Repository-Stand sämtliche Muss-Anforderungen erfüllt. Innerhalb des Muss-Umfangs haben Integrität und menschliche Kontrolle Vorrang vor Komfort; optionaler Zusatzkontext darf dagegen ausfallen, wenn die geringere Quellenbasis erkennbar bleibt.

Damit wird Priorität nicht mit Implementierungsreihenfolge gleichgesetzt. Sie bezeichnet die Bedeutung für den belastbaren Zielprozess und schafft den Maßstab, an dem unvollständige prototypische Umsetzung transparent bewertet werden kann.

Die Nummerierung der Anforderungen stellt Nachverfolgbarkeit her. Kapitel 4 ordnet Architekturgrenzen zu, Kapitel 5 beschreibt den Ist-Stand und Kapitel 6 bewertet Nachweise und Abweichungen. Ein Test belegt nur die tatsächlich geprüfte Eigenschaft: Ein bestandener Test für eine Inline-Mention bestätigt das gegenwärtige Parser-Verhalten,

widerlegt aber die Beschränkung aus FA-1. Ebenso sichern Schema- und Formattertests Struktur und Darstellung, nicht Faktentreue oder praktische Nutzbarkeit. Die Anforderungsmatrix legt damit fest, welche Evidenz eine Aussage tragen kann, und verbindet Literatur, Entwurf und Evaluation.

3.4. Qualitätsmodell für Ticketvorschläge

Ein Ticket gilt in dieser Arbeit als entwicklungsfähig, wenn sein Gegenstand verstanden, sein Umfang abgegrenzt, die Umsetzung gegen beobachtbare Kriterien geprüft und verbleibende Unsicherheit erkannt werden kann. Tabelle 3.2 fasst die daraus abgeleiteten fünf Informationsgruppen zusammen.

Tabelle 3.2: Informationsgruppen des abgeleiteten Qualitätsmodells

Informationsgruppe	Inhalt und Zweck
Ziel und Gegenstand	Eindeutiger Titel, Problem oder gewünschter Nutzen sowie betroffene Rollen oder Systeme
Kontext und Scope	Relevante Voraussetzungen, Randbedingungen, Abhängigkeiten und bewusste Nicht-Ziele
Verhalten und Kriterien	Erwartetes Ergebnis, bei Fehlern Ist-/Soll-Verhalten und Reproduktion, außerdem prüfbare Akzeptanzkriterien
Evidenz und Hinweise	Belegte Logs, Fehlermeldungen, Plattform- und Repository-Informationen ohne erfundene Implementierungsdetails
Unsicherheit	Offene Fragen, Widersprüche, Risiken, Annahmen und erforderliche Validierungsschritte

Das fachliche Zielmodell sieht neben Titel und Beschreibung auch Quellenbasis, Scope, Akzeptanzkriterien, technische Hinweise, offene Fragen, Risiken und Validierungsschritte vor. Es ist absichtlich breiter als ein Minimalformat, weil eine formal vollständige Ausgabe fehlende Informationen nicht verdecken darf. Kann ein Bereich nicht belastbar gefüllt werden, muss er als offen gekennzeichnet werden. Die spätere Implementierung verwendet ein kompakteres Ausgabeschema mit sechs Feldern; diese Abweichung wird in Kapitel 6 ausdrücklich bewertet.

Für die Rollen ergeben sich unterschiedliche Nutzungskriterien. Meldende Personen dürfen nicht voraussetzen müssen, dass sie interne Dateien oder Ursachen kennen. Rückfragen sollen deshalb fachliches Ziel, beobachtbares Verhalten und Erfolgskriterien betreffen. Product Owner benötigen eine klare Abgrenzung und sichtbare Annahmen. Entwicklerinnen, Entwickler und Qualitätssicherung benötigen einen Entwurf, aus dem Umsetzung und Prüfung getrennt hervorgehen. Für Betreiber sind Authentifizierung, Konfiguration, Logging und ein sicherer Fehlerzustand entscheidend. Diese Perspektiven erklären, warum die Qualität des Vorschlags nicht allein an sprachlicher Glätte gemessen werden kann.

3.5. Funktionale Anforderungen

Die funktionalen Anforderungen beschreiben den kontrollierten Kernworkflow. Ihre Nummerierung wird in der Evaluation wieder aufgegriffen.

FA-1 Explizite Aktivierung. Dev-Assist reagiert nur, wenn die erste inhaltliche Zeile einer Issue-Beschreibung oder eines Kommentars mit `@dev-assist` beginnt. Führende Leerzeichen und einfache Markdown-Präfixe dürfen toleriert werden.

FA-2 Relevante Ereignisse. Issue- und Note-Payloads müssen in ein gemeinsames internes Format mit Projekt, Issue, Text, Autor und Aktion überführt werden.

FA-3 Kommandotrennung. Eine normale Aktivierung startet die Analyse; `@dev-assist publish` startet ausschließlich die Übernahme eines zuvor gespeicherten Vorschlags.

FA-4 Abruf des Ticketkontexts. Soweit verfügbar, sind das aktuelle Issue, der Kommentarverlauf und optional begrenzter Repository-Kontext einzubeziehen. Webhook-Daten dienen als Fallback.

FA-5 Gemeinsame Analyse. Titel, Beschreibung, Kommentare, erkannte Antworten und Zusatzkontext sind gemeinsam auszuwerten. Gesicherte Aussagen, technische Hinweise und offene Punkte müssen unterscheidbar bleiben.

FA-6 Gezielte Rückfragen. Fehlen wesentliche fachliche Angaben, stellt das System konkrete, einzeln beantwortbare Fragen zu Ziel, Scope, erwartetem Verhalten oder Prüfkriterien. Bereits beantwortete Fragen sollen nicht wiederholt werden.

FA-7 Strukturierter Vorschlag. Bei ausreichendem Kontext erzeugt Dev-Assist einen als Issue-Beschreibung nutzbaren Entwurf. Unbelegte Inhalte dürfen nicht als Tatsachen dargestellt werden.

FA-8 Maschinenlesbares Ausgabeschema. Die Modellantwort muss einem definierten JSON-Schema entsprechen und vor jeder weiteren Verarbeitung geparkt und strukturell validiert werden.

FA-9 Sichtbare Rückgabe. Rückfragen, Vorschläge und Analysefehler werden als gekennzeichnete GitLab-Kommentare ausgegeben.

FA-10 Persistenz. Der analysierte Stand wird unter einem reproduzierbaren, gegen Pfadmanipulation begrenzten Speicherpfad als Markdown und optionale Metadaten gespeichert.

FA-11 Kontrollierter Publish. Publish liest ausschließlich den gespeicherten Stand, aktualisiert zuerst Titel und Beschreibung und bereinigt erst nach erfolgreicher Änderung Dev-Assist-bezogene Kommentare. Ohne erfolgreiche Analyse findet keine Übernahme statt.

3.6. Nicht-funktionale Anforderungen

Die nicht-funktionalen Anforderungen begrenzen Betrieb und Qualität. **NFA-1** fordert getrennte Module für Transport, GitLab, Analyse, Formatierung, Persistenz und Publish. **NFA-2** verlangt lokale Testbarkeit ohne reales GitLab und ohne kostenpflichtigen Modellaufruf. **NFA-3** betrifft Fallbacks und einen sicheren Abbruch bei ungültigem Kontext oder Modelloutput. **NFA-4** fordert nachvollziehbare Konsolenlogs ohne Secrets.

NFA-5 begrenzt KI-Risiken durch feste Anweisungen, Schema-Validierung, sichtbare Unsicherheit und menschliche Freigabe. **NFA-6** fordert standardmäßig verpflichtende Webhook-Authentifizierung und Replay-Schutz; unsicherer Betrieb darf nur ausdrücklich für lokale Entwicklung aktiviert werden. **NFA-7** verhindert Bot-Schleifen und kurzfristige Doppelverarbeitung. **NFA-8** trennt eine schnelle 2xx-Webhook-Antwort von der längeren Hintergrundverarbeitung. **NFA-9** verlangt Konfiguration über Umgebungsvariablen und die Trennung von Secrets und Quellcode. **NFA-10** erhält die Bedienung vollständig im GitLab-Workflow und verlangt keine zusätzliche Oberfläche.

Die Anforderungen werden für den Proof of Concept als Muss-Anforderungen behandelt, soweit sie den Kernprozess und seine Sicherheitsgrenzen betreffen. Erweiterte Bildanalyse, zusätzliche GitLab-Metadaten und differenzierte Fehlerdiagnosen sind Soll-Anforderungen. Dashboard, quantitative Produktmetriken und ein erweitertes Rollenmodell sind mögliche spätere Funktionen.

3.7. Abgrenzung und Zwischenfazit

Dev-Assist implementiert keinen Produktivcode, erstellt keine Merge Requests und priorisiert keine Tickets. Er ersetzt weder fachliche Entscheidungen noch die Prüfung eines Vorschlags. Die Arbeit bewertet die Struktur und technische Verarbeitung von Ticketinformationen, nicht die Qualität des gesamten Projekts. Eine eigenständige OCR- oder Vision-Komponente ist nicht Teil des Prototyps. Semantische Wahrheit kann durch das System nicht garantiert werden.

Damit verbindet die Anforderungsanalyse zwei Ebenen: Fachlich müssen Ziel, Kontext, Verhalten, Kriterien, Evidenz und Unsicherheit sichtbar sein. Technisch müssen Ereignisse authentifiziert, Kontext und Modelloutput begrenzt, Ergebnisse nachvollziehbar persistiert und Änderungen von einer menschlichen Freigabe abhängig gemacht werden. Diese Verbindung bildet den roten Faden der Architektur.

4. Konzeption der Systemarchitektur

Die Architektur überführt die Anforderungen in einen modularen Webhook-Dienst. Ihr zentrales Prinzip ist die Trennung zwischen probabilistischer Analyse und deterministischer Prozesskontrolle. Das Modell darf Inhalte strukturieren und Vorschläge formulieren; die Anwendung kontrolliert Ereignisse, Datenzugriffe, Antwortformat, Persistenz und produktive Änderungen.

4.1. Architekturziele und Komponenten

Die Architektur verfolgt vier Ziele: klare Verantwortlichkeiten, ereignisbasierte Integration, kontrollierte Autonomie und Nachvollziehbarkeit. Getrennte Module erleichtern Tests und Änderungen (vgl. Bass et al., 2021, Kap. 1). Webhooks vermeiden zyklisches Polling und koppeln GitLab lose an die nachgelagerte Verarbeitung (vgl. Hohpe/Woolf, 2003, Abschnitte "Messaging" und "Message Channel"). Ein Vorschau-/Publish-Muster begrenzt den Handlungsspielraum des Modells. Logging und persistierte Kontextdateien machen Übergaben nachvollziehbar.

Tabelle 4.1: Architekturbereiche, Verantwortlichkeiten und zentrale Module

Bereich	Verantwortung	Zentrale Module
HTTP und Webhooks	Middleware, Routen, Rohkörper, Authentifizierung, Parsing und schnelle Antwort	<code>src/app.ts</code> , <code>src/routes/</code> , <code>src/services/gitlab/auth.ts</code> , <code>src/services/gitlab/parser.ts</code>
GitLab und Kontext	Issues und Notes lesen oder ändern, Bot-Filter, Löschen der Bot-Kommentare und Repository-Daten	<code>src/services/gitlab/</code> , <code>src/services/repositorySummary.ts</code>
Analyse und Ausgabe	Prompt, Mock/OpenCode, Schema, Rückfragen und Markdown	<code>src/services/ai/</code> , <code>opencode.json</code>
Prozess und Persistenz	Analyse orchestrieren, Kontextdateien verwalten und Publish ausführen	<code>src/services/processing/</code> , <code>src/services/context/</code>

Tabelle 4.1 ordnet die Verantwortlichkeiten den später untersuchten Modulen zu. Abbildung 4.1 ergänzt diese statische Zuordnung um die beteiligten Plattformen, Datenflüsse und Vertrauensgrenzen.

Vier Vertrauensbereiche sind zu unterscheiden. GitLab verwaltet das fachlich verbindliche Issue und erzeugt externe Ereignisse. Der lokale Dev-Assist-Host enthält die deterministische Anwendung, den GitLab-Client und die Kontextpersistenz. OpenCode bildet eine gesonderte Ausführungsgrenze für den Analyseagenten; ein konfigurierter Modellanbieter liegt außerhalb des lokalen Hosts. Jeder rote Übergang in Abbildung 4.1 verändert deshalb die Vertrauensannahmen: Webhook-Daten müssen vor der Verarbeitung authentifiziert und validiert, Modellanfragen auf zulässigen Kontext begrenzt und zurückgegebene Inhalte als nicht vertrauenswürdige Vorschläge geprüft werden. Produktive Änderungen in GitLab dürfen ausschließlich über die kontrollierte Prozessschicht erfolgen.

Die lokale Kontextpersistenz liegt zwar innerhalb des Dev-Assist-Hosts, ist dadurch aber nicht automatisch sicher. Projekt- und Issue-Kennungen stammen aus externen Ereignissen oder manuellen Aufrufen und beeinflussen die Auswahl der Speicherpfade. Entsprechend muss die Pfadbildung bereits an der Eingangsgrenze auf erlaubte

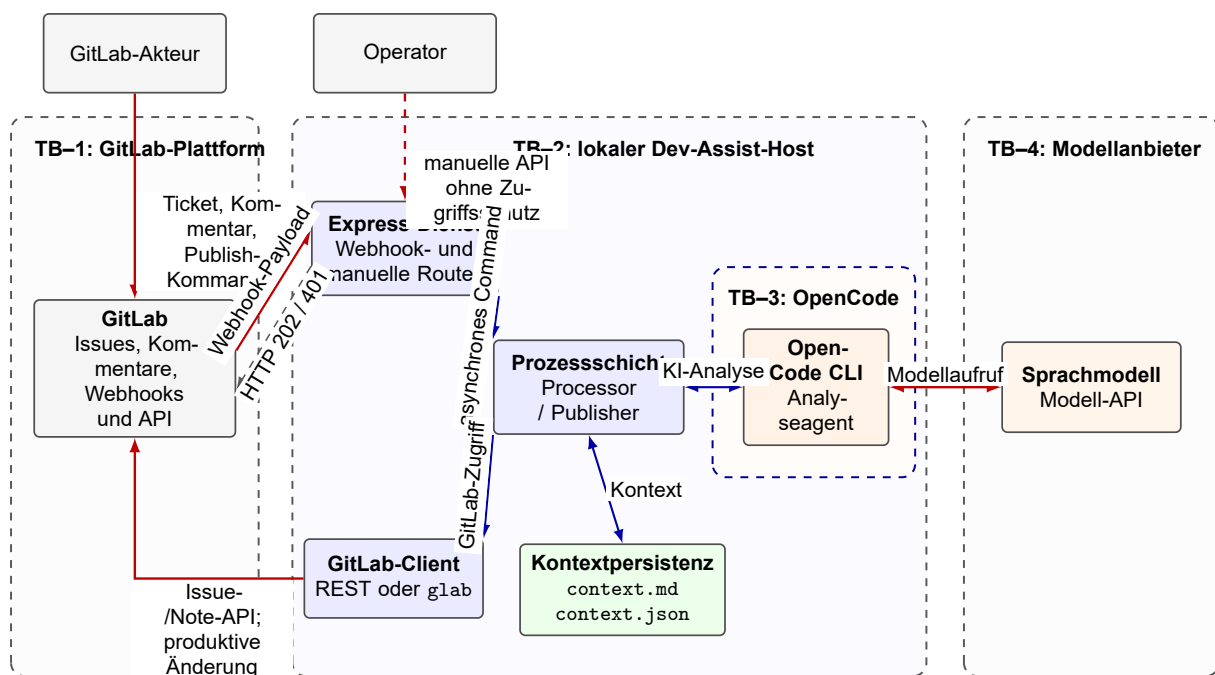


Abbildung 4.1.: Systemkontext und Komponenten des Dev-Assist-Prototyps mit Vertrauensgrenzen. Der Express-Dienst orchestriert GitLab-Zugriffe, KI-Analyse und lokale Kontextpersistenz. Rote Datenflüsse überschreiten eine Plattform-, Prozess- oder Anbietergrenze. Eigene Darstellung auf Grundlage des untersuchten Quellcodes, Stand 31.07.2026.

positive Kennungen und ein festes Wurzelverzeichnis begrenzt werden. Auch der gestrichelt dargestellte Operatorzugang benötigt eine eigene Authentifizierung, weil er dieselbe Analyse- und Publish-Logik ohne den GitLab-Webhook durchläuft. Das Diagramm macht damit sichtbar, dass Zugriffsschutz, Datenvalidierung und minimale Berechtigungen nicht einer einzelnen Komponente zugeordnet werden können, sondern an jeder überschrittenen Grenze zusammenwirken müssen.

Routen delegieren an die Prozessschicht; diese nutzt GitLab-, Analyse-, Repository- und Kontextdienste. Die KI-Schicht kennt keine GitLab-Schreiboperation. Kleine Parser-, Schema- und Formatterfunktionen sowie der Mock-Provider lassen sich isoliert prüfen; für die vollständige Prozessschicht sieht die aktuelle Umsetzung jedoch keine durchgängige Dependency Injection vor.

Zwei Alternativen wurden bewusst nicht gewählt. Ein periodisch abfragender Dienst hätte keine Webhook-Authentifizierung benötigt, würde aber zusätzliche Last, Verzögerungen und Zustandsverwaltung verursachen. Ein autonomer Agent mit direkten GitLab-Werkzeugen könnte weniger Anwendungscode benötigen, würde jedoch Modellentscheidung und produktive Änderung koppeln. Für den Untersuchungsgegenstand ist die gewählte Architektur geeigneter, weil Ereignisauslösung, Modellanalyse und Freigabe getrennt beobachtet und getestet werden können.

Auch Analyse und Publish werden im Zielbild nicht in einem einzigen Request zusammengeführt. Eine sofortige Übernahme könnte die Bedienung verkürzen, würde aber eine fehlerhafte Modellantwort unmittelbar wirksam machen. Der gespeicherte Zwischenstand soll genau dem zuvor sichtbaren Vorschlag entsprechen und einen zweiten Modelllauf während Publish verhindern. Kapitel 5 prüft, ob diese Konsistenz im Ist-Stand tatsächlich abgesichert ist.

Die Architekturentscheidungen erzeugen bewusst Zielkonflikte. Eine frühe HTTP-Antwort verbessert die Robustheit gegenüber Webhook-Timeouts, verlagert die fachliche Fehlerbehandlung jedoch in Logging und Kommentare. Lokale Persistenz macht den Übergang zu Publish reproduzierbar, benötigt dafür aber Schutz- und Aufbewahrungsregeln. Ein striktes Schema erleichtert Tests und verhindert beliebige Ausgabeformen, kann zugleich fachlich relevante Kategorien ausblenden. Repository-Kontext erhöht die technische Plausibilität, erweitert aber Datenumfang und Angriffsfläche.

Diese Trade-offs werden anhand der Anforderungen priorisiert. Prozesskontrolle und Datenintegrität haben Vorrang vor einer möglichst autonomen Bedienung. Reproduzierbarkeit hat im Proof of Concept Vorrang vor horizontaler Skalierbarkeit. Ein kompaktes, streng geprüftes Ausgabeschema wird zunächst einer flexiblen, schwer validierbaren Ausgabe vorgezogen. Wo diese Entscheidungen den fachlichen Nutzen begrenzen, werden sie als offene Grenze dokumentiert. Die Architektur beansprucht daher nicht, alle Qualitätsziele gleichzeitig zu maximieren. Sie schafft eine untersuchbare Grundlage, auf der technische Sicherheit und semantische Wirksamkeit getrennt evaluiert werden können.

4.2. Bewertung zentraler Architekturentscheidungen

Webhooks machen das auslösende Ereignis zum expliziten Verarbeitungseingang und unterstützen die Nachverfolgbarkeit von Aktivierung und Kommando. Gegenüber Polling verringern sie Verzögerung und zusätzlichen Zustandsbedarf, schaffen aber eine öffentliche Grenze für Authentizität, Wiederholung und unvollständige Payloads. Für den interaktiven Issue-Workflow überwiegt dieser Vorteil.

Transportbestätigung und Analyse werden getrennt, damit GitLab vor dem längeren Modellaufruf einen 2xx-Status erhält. Synchrone Verarbeitung könnte das fachliche Ergebnis im HTTP-Status abbilden, würde jedoch Timeouts und erneute Lieferungen begünstigen. Eine dauerhafte Queue wäre produktiv robuster; der Prototyp nutzt nur Hintergrundausführung und In-Memory-Deduplizierung. Er bietet deshalb weder Zustellungsgarantie noch horizontale Skalierbarkeit.

Die frühe Antwort verschiebt Verantwortung auf nachgelagerte Beobachtung: Fehler müssen über Logs oder gekennzeichnete Kommentare erkennbar werden. Ohne dauerhaften Auftragsspeicher bleibt dieser Nachweis jedoch unvollständig.

Die lokale Markdown-Persistenz hält den Vorschlag ohne weitere Infrastruktur prüfbar, besitzt aber keine Transaktionen, Versionierung oder zentrale Aufbewahrungsregeln. Publish darf daher nur den gespeicherten Stand übernehmen und keinen neuen Modelllauf auslösen. Fehler werden nach Wirkung behandelt: Fehlender Zusatzkontext erlaubt einen gekennzeichneten Fallback, ungültiger Modelloutput keinen regulären Publish-Kontext. Bei Änderungen in GitLab ist die Issue-Aktualisierung die Kernwir-

kung; das Löschen der Bot-Kommentare muss im Soll erst danach soweit möglich erfolgen.

Dieses Kapitel beschreibt den Maßstab, nicht den erreichten Sicherheitsstand. Die Implementierung wird daran geprüft. Dadurch bleiben permissive Mention-Erkennung, optionale und von den aktuellen Signaturvorgaben abweichende Signaturprüfung, ungeschützte manuelle Routen, ungeprüfte Pfadsegmente und umgekehrte Publish-Reihenfolge als Abweichungen sichtbar und führen von der Konzeption zur Evaluation.

4.3. Ereignis- und Datenfluss

Im architektonischen Zielablauf erreicht ein GitLab-Webhook `/webhooks/gitlab/issues`. Die Middleware bewahrt den unveränderten Body für die Signaturprüfung. Der Router soll anschließend Authentizität, Ereignisform, Bot-Ursprung, Mention, Kommando und Duplikat prüfen. Akzeptierte Ereignisse erhalten HTTP 202; die längere Verarbeitung läuft nachgelagert. Diese Trennung entspricht der GitLab-Empfehlung, Webhook-Receiver schnell antworten zu lassen (vgl. GitLab Documentation, o. J.a, Abschnitt "Webhook receiver requirements").

Im Analysepfad lädt der Processor nach Möglichkeit das aktuelle Issue und die Notes. Scheitert dieser Abruf, bleibt der Webhook-Text als begrenzter Fallback verfügbar. Optional ergänzt eine Repository-Zusammenfassung technische Hinweise. Aus diesen Daten entsteht ein Analysekontext, der gemeinsam mit festen Anweisungen an den OpenCode-Agenten übergeben wird.

Die Antwort wird aus dem CLI-Output extrahiert, als JSON geparkt und gegen das Ausgabeschema geprüft. Ungültiger Output erzeugt keinen Publish-Kontext. Eine gültige Analyse wird je nach offenen Fragen als Klärungskommentar oder strukturierter Vorschlag gerendert und in `context.md` sowie `optional context.json` gespeichert.

Publish ist im Zielbild ein eigener Datenfluss. Der Publisher liest den gespeicherten Stand, bereitet Titel und Beschreibung auf und soll zunächst das Issue über die GitLab API aktualisieren. Erst danach sollen eindeutig Dev-Assist-bezogene Kommentare best effort bereinigt werden. Damit zerstört ein fehlgeschlagener Issue-Write nicht den vorherigen Diskussionsverlauf. Das Modell wird im Publish-Pfad nicht erneut aufgerufen.

Abbildung 4.2 stellt dem Zielbild den tatsächlich untersuchten Webhook-Pfad gegenüber. Sie verbindet Normalfall, Fehlerpfade und Persistenz in einer gemeinsamen zeitlichen Sicht.

Die frühe Antwort mit HTTP 202 bestätigt ausschließlich die technische Annahme des Ereignisses. Sie bestätigt weder eine erfolgreiche Kontextbeschaffung noch einen gültigen Modelloutput oder eine spätere Issue-Änderung in GitLab. Nach dieser Transportgrenze besitzt der Webhook-Aufrufer keinen synchronen Rückkanal zum fachlichen Ergebnis. Fehler werden nur über einen nachgelagerten Kommentar oder die Protokollierung sichtbar; ohne dauerhafte Queue und expliziten Auftragsstatus kann ein Prozessabbruch daher nicht zuverlässig von einer noch laufenden Verarbeitung unterschieden werden. Für einen Proof of Concept ist diese Entkopplung einfach umsetzbar, für einen produktiven Betrieb wären persistente Aufträge, Korrelation und wiederholbare Zustandsübergänge erforderlich.

Im Analysepfad sind Kommentar und Kontextdatei zwei getrennte Seiteneffekte. Scheitert das Schreiben der GitLab-Note, kann die vollständige Analyse dennoch lokal gespeichert werden. Umgekehrt wird auch bei einem sichtbaren Klärungskommentar der vollständige Modellvorschlag persistiert. Damit ist der gespeicherte Stand kein zuverlässiger Nachweis dessen, was die freigebende Person gesehen hat. Im Publish-Pfad verschärft die aktuelle Reihenfolge diese Lücke: Der Publisher liest den Vollvorschlag, entfernt zunächst ausgewählte Kommentare und aktualisiert erst danach das Issue. Die Sequenzdarstellung begründet deshalb zwei getrennte Anforderungen an eine Korrektur: eine revisionsgebundene Freigabe vor der Issue-Änderung und eine fehlersichere Reihenfolge, bei der die Kernänderung bestätigt sein muss, bevor erläuternde Kommentare gelöscht werden.

Die Fehlerbehandlung folgt der Bedeutung eines Schritts. Lesefehler beim Abruf zusätzlicher Notes reduzieren den Kontext, können aber mit einem klaren Fallback noch eine gekennzeichnete Analyse erlauben. Ein Parsing- oder Schemavalidierungsfehler verhindert dagegen die Erzeugung eines regulären Vorschlags. Beim Publish führen fehlender Kontext oder ein fehlgeschlagenes Issue-Update zum Abbruch. Tabelle 4.2 macht die unterschiedliche Behandlung reversibler und irreversibler Wirkungen explizit.

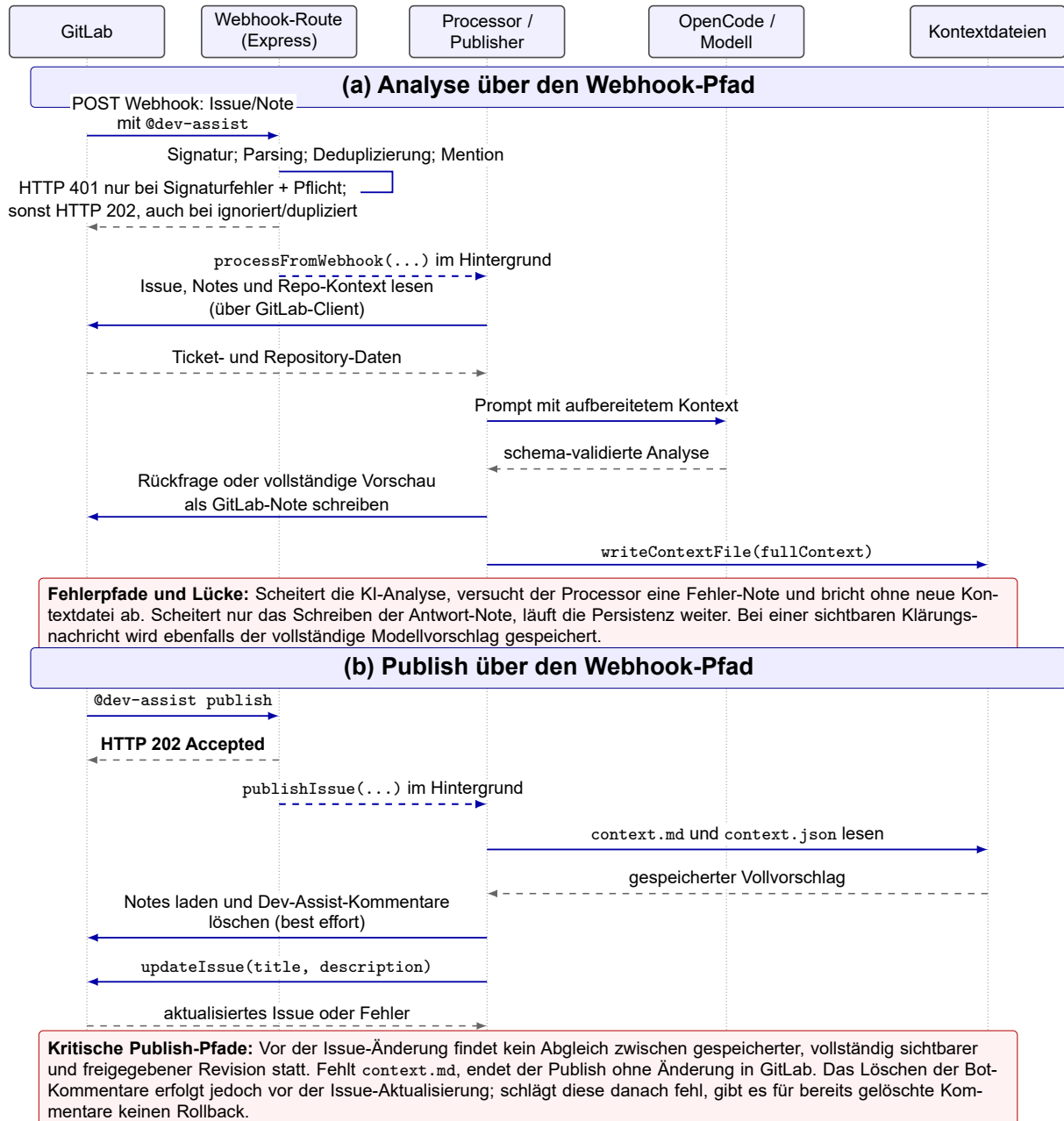


Abbildung 4.2.: Sequenz der Analyse- und Publish-Verarbeitung über den GitLab-Webhook. Die Route bestätigt angenommene Ereignisse früh mit HTTP 202 und führt Process beziehungsweise Publish anschließend ohne Rückkanal zum Aufrufer aus. Rote Hinweise kennzeichnen implementierte Fehlerpfade und die fehlende Kopplung zwischen sichtbarer Vorschau, gespeichertem Kontext und Freigabe. Eigene Darstellung auf Grundlage des untersuchten Quellcodes, Stand 31.07.2026.

Tabelle 4.2: Fehlerklassen und vorgesehene Reaktionen im Zielablauf

Fehlerklasse	Systemreaktion	Begründung
Ungültiger Webhook	Ablehnung vor fachlicher Verarbeitung	Externe Eingänge dürfen keine internen Schritte auslösen.
Fehlender Zusatzkontext	Warnung und begrenzter Fallback	Ein Vorschlag kann möglich bleiben, muss aber Kontextlücken erkennen lassen.
Ungültige Agentenantwort	Kein normaler Kontext; Fehler wird sichtbar gemacht	Strukturell unkontrollierter Modelltext darf nicht publishbar werden.
Fehlgeschlagenes Issue-Update	Publish-Abbruch, Kommentare bleiben erhalten	Die bestehende Diskussion darf nicht vor der dauerhaften Änderung verloren gehen.
Fehler beim Löschen der Bot-Kommentare	Protokollierung nach erfolgreichem Update	Die Kernänderung ist bereits erfolgt; das Löschen bleibt nachgeordnet.

4.4. Prompt, Ausgabeschema und Kontextgrenzen

Der Prompt trennt feste Regeln von untrusted Ticketdaten. Er fordert Ziel, Nutzerwert, Beschreibung, Akzeptanzkriterien, belegten technischen Kontext, Lösungsvorschlag und offene Fragen. Repository-Informationen dürfen technische Hinweise stützen, aber keine fachlichen Anforderungen erfinden. Antworten auf frühere Rückfragen werden hervorgehoben, damit das Modell sie in eine erneute Analyse einbezieht.

Das implementierte Ausgabeschema enthält exakt `title`, `description`, `acceptanceCriteria`, `technicalContext`, `proposedSolution` und `openQuestions`. Dieses kompakte Schema ist leichter deterministisch zu validieren als freier Fließtext, bildet das breitere fachliche Zielmodell aus Kapitel 3 jedoch nicht vollständig ab. Architektur und Evaluation behandeln diese Differenz als sichtbaren Trade-off, nicht als stillschweigende Gleichsetzung.

Die Repository-Zusammenfassung ist begrenzt auf Projektmetadaten, Sprachen, einen Ausschnitt des Dateibaums und ausgewählte Schlüsseldateien. Sie wird gecacht und kann veralten. Bilder und Screenshots werden nicht heruntergeladen oder inhaltlich

analysiert. Für eine spätere Erweiterung wären Dateityp- und Größenprüfung, Zugriffsschutz, OCR/Vision, Kennzeichnung unsicherer Extraktion und Schutz gegen indirekte Prompt Injection erforderlich.

4.5. Sicherheit, Robustheit und Testbarkeit

Das Sicherheitskonzept fordert das Signaturverfahren von GitLab, einen konstantzeitlichen Vergleich und ein zulässiges Zeitfenster gegen Replay-Angriffe. Die Signaturpflicht soll standardmäßig aktiv sein; ein toleranter Modus ist ausschließlich für lokale Entwicklung vorgesehen. Manuelle Process-/Publish-Routen sollen durch einen separaten Bearer-Token geschützt und ohne diese Konfiguration deaktiviert sein.

Mention-Gate, Bot-Filter und Deduplizierung sollen unbeabsichtigte Aktivierungen und unmittelbare Schleifen verhindern. Projekt- und Issue-IDs müssen vor der Pfadbildung auf positive Ganzzahlen begrenzt werden. Bei Lesefehlern kann die Analyse mit weniger Kontext fortgesetzt werden; bei ungültigem Modelloutput oder fehlender Kontextdatei darf dagegen kein regulärer Publish ausgeführt werden. Secrets liegen in Umgebungsvariablen und dürfen nicht geloggt werden.

Die Architektur soll externe Systeme durch Interfaces und Provider trennen. Der Mock-Provider liefert deterministische Analysen; GitLab-Client, OpenCode-Laufzeit und Repository-Summary sollen für Komponententests ersetzbar sein. Die Prüfung konzentriert sich auf Parsing, Authentifizierung, Schema, Formatierung, Persistenz und Publish-Reihenfolge. Kapitel 5 zeigt, inwieweit der Prototyp dieses Testbarkeitsziel erreicht.

4.6. Zwischenfazit

Dev-Assist verbindet GitLab-Webhooks und APIs mit einer gekapselten OpenCode-Analyse und einem kontrollierten Publish-Prozess. Explizite Aktivierung, begrenzte Kontextauswahl, festes Ausgabeschema, lokale Persistenz und menschliche Freigabe bilden gemeinsam die Sicherheits- und Nachvollziehbarkeitsgrenze. Kapitel 5 zeigt, wie diese Architektur im TypeScript-Prototyp umgesetzt ist.

5. Implementierung

Dieses Kapitel beschreibt die Umsetzung des Prototyps entlang des tatsächlichen Repository-Stands. Zunächst wird der funktionierende Kernworkflow dargestellt. Anschließend folgen Projektaufbau, Schnittstellen und die Stellen, an denen der Ist-Stand von den Muss-Anforderungen abweicht. Kleinteilige Dateibeschreibungen werden nur genannt, wo sie für Nachvollziehbarkeit und Evaluation erforderlich sind.

5.1. Funktionierender Kern

Der Prototyp setzt die Kette von der GitLab-Ereignisannahme bis zur getrennten Veröffentlichung als ausführbaren Dienst um. Tabelle 5.1 trennt vorhandene Kernfunktionen von den später bewerteten Abweichungen, damit der Nachweis nicht nur aus Lücken besteht.

Tabelle 5.1: Leistungsstand der Kernfunktionen im untersuchten Prototyp

Kernfunktion	Umgesetzter Stand
Webhook-Annahme	Issue- und Note-Ereignisse werden geparkt; akzeptierte Lieferungen erhalten HTTP 202, die Analyse läuft danach.
Kontextaufbereitung	Aktuelles Issue, Notes und optional begrenzter Repository-Kontext werden vor der Analyse zusammengeführt.
Modellgrenze	OpenCode oder Mock liefern eine Antwort; nur das JSON-Schema mit sechs Feldern wird akzeptiert.
Rückfrage oder Vorschlag	Mindestens zwei substanzielle offene Fragen erzeugen einen Klärungskommentar, sonst einen strukturierten Entwurf.
Persistenz und Publish	Der analysierte Stand wird lokal gespeichert; <code>@dev-assist publish</code> übernimmt ihn ohne neuen Modelllauf.
Nachweisbare Teilprüfungen	Parser, Schema, Formatter und ausgewählte Routen sind durch 60 Tests abgesichert.

Dieser Kern ist lauffähig und lokal ohne reales Modell testbar. Er erfüllt noch nicht den vollständigen Kontrollanspruch aus Kapitel 3: Die Aktivierung ist weiter als FA-1, die Signaturprüfung weicht von NFA-6 ab, manuelle Routen und Speicherpfade sind ungeschützt, und Publish bindet weder sichtbare Revision noch die geforderte Reihenfolge der Issue-Änderung. Die folgenden Abschnitte beschreiben zuerst die technische Realisierung dieses Kerns und danach diese Abweichungen.

5.2. Projektaufbau und Laufzeitkonfiguration

Dev-Assist ist eine Node.js-/TypeScript-Anwendung im ECMAScript-Modulformat. `npm run dev` startet den Entwicklungsserver mit `tsx`, `npm run build` kompiliert nach `dist/`, und `npm test` führt den Node.js-Test-Runner mit TypeScript-Import aus. `tsconfig.json` verwendet `NodeNext`, `ES2022` und `strict: true`. Die statische Typprüfung unterstützt konsistente interne Datenformen, wird an externen Grenzen jedoch durch Laufzeitprüfungen ergänzt.

`src/server.ts` initialisiert Konfiguration, Logging und optional einen Cloudflare-Tunnel. `src/app.ts` registriert Middleware und Routen. Fachliche Dienste liegen unter `src/services/`; Tests befinden sich unter `tests/`. Die OpenCode-Agenten werden in `open code.json` konfiguriert. Tabelle 5.2 ordnet diese Struktur den umgesetzten Verantwortlichkeiten zu.

`src/config.ts` lädt die nicht versionierte `.env` und normalisiert unter anderem Port, Log-Level, GitLab-URL, Webhook-Secret, Signaturpflicht, Provider, Modell, Timeout, Kontextverzeichnis und Deduplizierungsfenster. `AI_PROVIDER=mock` ermöglicht einen deterministischen Betrieb ohne externes Modell; `AI_PROVIDER=opencode` aktiviert die CLI. GitLab-Zugriffe erfolgen primär über ein bereits authentifiziertes `glab`, alternativ über REST und Personal Access Token.

Tabelle 5.2: Projektbereiche und umgesetzte Verantwortlichkeiten

Projektbereich	Umgesetzte Verantwortung
<code>src/routes/</code>	Health-Check, Webhook-Eingang sowie derzeit ungeschützte manuelle Process-/Publish-Endpunkte
<code>src/services/gitlab/</code>	Signatur, Payload-Parsing, Mention, Kommando, GitLab-Client und Löschen der Bot-Kommentare
<code>src/services/ai/</code>	Prompt, Mock-/OpenCode-Aufruf, Schema, Rückfragen und Markdown-Formatierung
<code>src/services/context/</code> , <code>src/services/processin</code> <code>g/</code>	Persistenz sowie Orchestrierung der Analyse- und Publish-Phase
<code>tests/</code>	Automatisierte Prüfung ausgewählter deterministischer Systemteile und Grenzfälle

Die Konfiguration steuert Betriebs- und Sicherheitsmodi, setzt das Zielbild jedoch nicht vollständig um. `GITLAB_REQUIRE_SIGNATURE` ist standardmäßig `false`; fehlt das Signing-Secret, akzeptiert die Prüffunktion Webhooks sogar dann, wenn das Flag aktiviert ist. Die manuellen Routen besitzen weder Bearer-Token noch Deaktivierung ohne Token. Für den OpenCode-Pfad prüft der Server die CLI-Verfügbarkeit. Der Standard ist damit nicht fail closed.

5.3. HTTP- und Webhook-Verarbeitung

Express liest JSON mit einem Größenlimit von 1mb. Eine `verify`-Funktion speichert den Rohkörper, bevor das Payload deserialisiert wird. Diese Reihenfolge ist notwendig, weil die Webhook-Signatur auf den empfangenen Bytes beruht. Eine Request-Middleware protokolliert Methode, Pfad, Ereignistyp, Status und Dauer.

Die Anwendung stellt `/health`, den Webhook-Endpunkt sowie ungeschützte manuelle Process-/Publish-Routen bereit. `auth.ts` liest `x-gitlab-signature` und `x-gitlab-timestamp`, berechnet eine hexadezimale HMAC-SHA256 über `timestamp:body` und versucht zusätzlich eine direkte Body-HMAC. Der Vergleich erfolgt bei gleicher Länge konstantzeitlich; eine Zeitstempelprüfung fehlt. HTTP 401 entsteht nur, wenn ein konfiguriertes Secret zu einer fehlgeschlagenen Prüfung führt und die Signaturpflicht aktiv

ist; ohne Secret meldet der Verifier Erfolg.

`parseGitLabWebhook` vereinheitlicht Issue- und Note-Events. Das resultierende Objekt enthält Projekt-ID, Issue-IID, Titel, Beschreibung beziehungsweise Kommentar, Aktion und Kommando, jedoch keinen Autor; FA-2 ist insoweit nur teilweise erfüllt. Entgegen FA-1 erkennt `mention.ts` `@dev-assist` an beliebiger Textposition. Bei Issue-Events genügt außerdem eine Mention im Titel; bei Note-Events wird nur der aktuelle Kommentar geprüft. Der Command-Parser unterscheidet `Analyse` und `publish`.

Eigene Bot-Kommentare und Ausgaben mit Dev-Assist-Markern werden ignoriert. Die In-Memory-Deduplizierung nutzt Projekt, Issue, Ereignisart und `event_id`, `object_attributes.id` oder `no-id`. Stabile Issue-IDs und der Ersatzwert können unterschiedliche Ereignisse im TTL-Fenster zusammenfassen. Zudem wird der Schlüssel vor `shouldProcess` gespeichert, sodass ein ignoriertes Ereignis ein später relevantes unterdrücken kann. Delivery-Header, Body-Hash, Neustarts und mehrere Instanzen sind nicht abgedeckt.

5.4. GitLab-Client und Prozesssteuerung

`src/services/gitlab/client.ts` kapselt Issue-, Notes- und Repository-Operationen. Im Standardpfad wird `glab api per execFile` ausgeführt und die JSON-Ausgabe geparkt. Der REST-Fallback sendet ein konfiguriertes Token. Dadurch bleibt die Prozesslogik unabhängig von CLI-Argumenten, URLs und HTTP-Headern. Der Client stellt unter anderem `getIssue`, `listNotes`, `createNote`, `deleteNote` und `updateIssue` bereit.

`processor.ts` orchestriert die Analyse. Es lädt Issue und Notes, verwendet bei Lese Fehlern den Webhook-Kontext und fordert optional eine Repository-Zusammenfassung an. Danach ruft es `ai.analyzeTicket` auf. Bei einem Analysefehler wird nach Möglichkeit ein Fehlerkommentar erzeugt; aus ungültigem Output entsteht keine normale Kontextdatei.

Bei mindestens zwei substanziellen offenen Fragen rendert der Processor einen Klärungskommentar, andernfalls einen vollständigen Vorschlag. Die Schwelle berücksichtigt nicht die Kritikalität einer einzelnen Frage. In beiden Fällen werden vollständiger

Analysekontext und Titel unabhängig vom sichtbaren Kommentar lokal gespeichert. Der Publisher prüft weder Klärungsstatus noch offene Fragen; damit kann ein Stand übernommen werden, der zuvor nicht als vollständiger Vorschlag sichtbar war.

Die GitLab-Abstraktion unterstützt zwei Betriebswege. `glab api` nutzt den bereits eingerichteten CLI-Kontext und vermeidet eine zweite Authentifizierungskonfiguration. Der REST-Fallback ermöglicht denselben Funktionsumfang über `PRIVATE-TOKEN`. Das gemeinsame Interface normalisiert Issue-, Note- und Repository-Operationen, sodass der Processor nicht zwischen Transportvarianten unterscheidet. Fehler werden an der Prozessgrenze kontextbezogen behandelt und mit Projekt, Issue und Phase geloggt.

Die manuellen HTTP-Routen und `src/cli/publish-issue.ts` verwenden dieselbe Prozesslogik wie der Webhook. Dadurch existiert kein zweiter fachlicher Implementierungspfad für lokale Wiederholungen. Die Routen übernehmen Projekt-ID und Issue-ID derzeit jedoch ungeprüft als Zeichenketten und sind nicht durch einen separaten Token geschützt. Zusätzliche Publish-Aktionen wie Labels oder Statuswechsel werden explizit übergeben. Ohne vorherigen Process-Schritt fehlt die erforderliche Kontextdatei, sodass Publish keinen neuen Inhalt improvisiert.

5.5. Durchgängiger Verarbeitungsfall

Der Zusammenhang der Komponenten wird an einem generischen Analyse-/Publish-Fall deutlich. Eine Person ergänzt eine Issue-Beschreibung oder einen Kommentar um `@dev-assist`; im Ist-Stand genügt jede Textposition und bei Issue-Events auch der Titel. GitLab liefert das Ereignis an den Webhook. Express bewahrt den Rohkörper, die optionale Authentifizierung prüft die übermittelten Signaturdaten nach dem implementierten Verfahren, und der Parser überführt die unterschiedlichen Issue- oder Note-Strukturen in dasselbe interne Format.

Nach Mention-, Bot- und Deduplizierungsprüfung antwortet die Route mit HTTP 202. Diese Antwort bestätigt nur die Annahme des Ereignisses; sie behauptet noch keinen erfolgreichen Modelllauf. Der Hintergrundprozess lädt das aktuelle Issue und die Notes. Sind externe Leseschritte nicht verfügbar, kennzeichnet das Logging den Fallback und verwendet die bereits enthaltenen Webhook-Daten. Eine verfügbare Repository-Zusammenfassung wird nur als begrenzter technischer Zusatz angefügt.

Der AI-Service erstellt aus festen Anweisungen und Arbeitsdaten den Prompt. Open-Code führt den vordefinierten Agenten aus. Die Anwendung extrahiert anschließend die finale Textantwort, parst sie und akzeptiert nur das sechsteilige Ausgabeschema. Mehrere substantielle offene Fragen führen zu einem Klärungskommentar. Bei ausreichender Grundlage entsteht ein Vorschlag mit Beschreibung, Akzeptanzkriterien, technischem Kontext und Lösungsskizze. In beiden Fällen bleibt sichtbar, dass Dev-Assist einen Entwurf und keine freigegebene Anforderung erzeugt.

Parallel zur sichtbaren Rückgabe speichert die Kontextschicht den analysierten Stand. Eine spätere Antwort kann in eine erneute Analyse einfließen; die Klärungslogik markiert erkannte Antworten und verhindert ähnliche Wiederholungsfragen. Erst ein gesonderter Kommentar mit `@dev-assist publish` wechselt in den Publish-Pfad. Dieser liest den vorhandenen Kontext, ohne das Modell erneut auszuführen. Abweichend vom Zielbild bereinigt er zugehörige Assistenzkommentare jedoch vor der Aktualisierung des Issues.

Dieser Fall zeigt den roten Faden der Implementierung: Externe Ereignisse werden technisch geprüft und fachlich gefiltert, dann probabilistisch analysiert und zuletzt wieder deterministisch verarbeitet. Die modulare Trennung macht die Übergänge grundsätzlich überprüfbar. Der Anforderungsabgleich zeigt zugleich, dass die Autorisierung, die Aktivierungsgrenze und die Reihenfolge der GitLab-Änderungen noch nachgeschärft werden müssen. Ein konstruierter Beispielverlauf, der dasselbe Ausgabeschema verwendet, ist in Anhang A dargestellt.

5.6. Zustand und Konsistenz an den Prozessgrenzen

Der Webhook-Router hält Deduplizierung nur prozesslokal. Der Schlüssel kombiniert Projekt, Issue, Ereignisart und `event_id` beziehungsweise `object_attributes.id`; andernfalls wird `no-id` verwendet. Die TTL-Map reduziert kurzfristige Wiederholungen, geht bei Neustarts verloren und wird nicht zwischen Instanzen geteilt. Der Ersatzwert kann zudem verschiedene Ereignisse desselben Issues zusammenfassen. Dies ist eine Schleifenbremse, keine Exactly-once-Garantie.

Nach HTTP 202 läuft die Verarbeitung in einer nicht abgewarteten asynchronen Funktion. Spätere Fehler werden protokolliert, aber weder dauerhaft eingereicht noch auto-

matisch wiederholt. Der Processor nutzt bei Lesefehlern Webhook-Daten als Fallback. Ab zwei substantiellen offenen Fragen erzeugt er einen Klärungskommentar, sonst einen Vorschlag. Diese deterministische Schwelle berücksichtigt nicht, ob eine einzelne Frage fachlich kritisch ist.

GitLab-Kommentar und lokale Persistenz bilden keine Transaktion. Ein fehlgeschlagener Kommentar verhindert das Schreiben von `context.md` nicht; umgekehrt kann ein sichtbarer Vorschlag ohne gespeicherten Publish-Stand entstehen. Im Klärungsfall enthält die Datei außerdem mehr als der sichtbare Kommentar, ohne dass Publish diesen Status sperrt. Eine Vorschlags-ID mit persistentem Status wäre erforderlich.

Der aktuelle Zustand ist deshalb nur aus mehreren Artefakten rekonstruierbar: GitLab zeigt die Kommunikation, die lokale Datei den publishbaren Stand und das Konsolenlog den Prozessverlauf. Eine automatische Konsistenzprüfung zwischen ihnen fehlt.

Der Publisher liest Kontext und Titelmetadaten, löscht Dev-Assist-Notes best effort und ruft erst danach `updateIssue` auf. Bei fehlgeschlagenem Update können Kommentare bereits entfernt sein; FA-11 ist nicht erfüllt. Zusätzliche Issue-Änderungen erfolgen separat und ebenfalls nicht atomar. Manuelle Routen übernehmen Projekt- und Issue-Werte außerdem ohne Bearer-Schutz und Positivzahlprüfung; Reader und Writer bilden daraus ungeprüfte Pfade. Diese Ist-Grenzen begründen den folgenden Anforderungsabgleich.

5.7. OpenCode, Prompt und Laufzeitvalidierung

`src/services/ai/service.ts` stellt für Mock und OpenCode dasselbe Analyseinterface bereit. `instructions.ts` enthält Persona, Regeln, Schema-Beispiel und Befüllungshinweise. `buildUserPrompt` kombiniert diese Vorgaben mit Issue, Kommentaren, erkannten Antworten und optionaler Repository-Zusammenfassung. Ticketinhalte werden ausdrücklich als zu analysierende Daten eingeordnet.

Der OpenCode-Pfad startet `opencode run` mit Agent, Modell und JSON-Ausgabeformat. Ein Timeout begrenzt die Laufzeit. `parseOpencodeAnalysisOutput` entfernt ANSI-Sequenzen, untersucht Gesamttext, eingebettete JSON-Objekte und JSONL-Ereignisse und nutzt bei Bedarf `opencode export`. Die Kandidaten werden in

Fundreihenfolge geprüft; der erste schema-konforme Wert wird akzeptiert.

`schema.ts` definiert das sechsteilige Ausgabeschema. `parseAnalysisJson` entfernt optionale Codezäune und parst JSON; `validateRequirementAnalysis` verlangt exakt die bekannten Felder, einen String-Titel und String-Arrays für alle übrigen Werte. Unbekannte Felder oder falsche Typen werden abgelehnt. Diese Prüfung ist streng gegenüber Struktur, nicht gegenüber Bedeutung.

`formatter.ts` erzeugt Rückfragen oder die sichtbaren Bereiche Description, Acceptance Criteria, Technical Context & Logs und Proposed Solution. Leere Arrays erhalten einen Hinweis auf fehlende Information; nicht leere Platzhalter wie `unknown` werden dagegen unverändert gerendert. `clarifications.ts` ordnet späteren Text früheren Fragen zu und filtert stark ähnliche Fragen. Die Heuristik ist deterministisch, aber semantisch begrenzt.

Die CLI-Integration behandelt Ausgabe und Prozesszustand getrennt. Standardausgabe kann JSONL-Ereignisse, Textfragmente oder nur eine Session-ID enthalten; Standardfehler und Exit-Code bleiben für Diagnosen erhalten. Ein Export-Fallback liest bei Bedarf den Session-Verlauf. Eine explizite Erkennung bekannter Schema-Beispiele oder eine Bevorzugung der letzten Antwort existiert nicht. Formal gültiges Beispiel-JSON im Output kann daher fälschlich als Analyse akzeptiert werden.

Schema und Formatter bilden gemeinsam die Anwendungsschnittstelle. Das Schema begrenzt Datentypen und Feldnamen, während der Formatter festlegt, wie diese Daten in GitLab erscheinen. Änderungen an Prompt, Schema und Darstellung können deshalb nicht unabhängig voneinander erfolgen. Das kompakte Schema erleichtert Tests, erzeugt aber den in Kapitel 3 beschriebenen fachlichen Verlust. Diese Spannung wird nicht durch zusätzliche Prompttexte gelöst, sondern als Gegenstand der Evaluation erhalten.

5.8. Kontext, Publish und Repository-Zusammenfassung

`src/services/context/` schreibt `.dev-assist/issues/<projectId>/<issueId>/context.md` und optional `context.json`. Projekt- und Issue-ID werden dabei ohne Positivzahl- oder Pfadsegmentprüfung an `path.resolve` übergeben. Externe Parame-

ter können den vorgesehenen Kontextbereich daher potenziell verlassen. Markdown bleibt für Menschen lesbar, während JSON den vorgeschlagenen Titel getrennt hält.

`publisher.ts` liest beide Dateien und bereitet die finale Issue-Beschreibung auf. Der Titel stammt bevorzugt aus den Metadaten, ersatzweise aus dem Markdown. Dev-Assist-bezogene Notes werden bestimmt und best effort gelöscht; normale Nutzerkommentare und Systemnotes bleiben erhalten. Erst danach ruft die Implementierung `updateIssue` auf. Schlägt dieses Update fehl, ist die vorherige Diskussion somit bereits teilweise entfernt. Diese Reihenfolge verletzt FA-11 und muss vor produktivem Einsatz umgekehrt werden.

`repositorySummary.ts` sammelt Projektmetadaten, Sprachen, höchstens 300 Baueinträge und bis zu acht ausgewählte Schlüsseldateien mit begrenzter Länge. Ein eigener Agent verdichtet sie zu einer technischen Zusammenfassung, die gecacht und lokal persistiert wird. Sie ergänzt den Prompt, ersetzt aber nicht die fachliche Ticketbeschreibung. Eine Bilddownload-, OCR- oder Vision-Pipeline ist nicht implementiert.

Die Begrenzung des Repository-Kontexts ist sowohl technisch als auch fachlich relevant. Ein vollständiges Repository würde Promptgröße, Laufzeit und potenziell übertragene vertrauliche Daten erhöhen. Die Auswahl bevorzugt deshalb Dateien, die Technologie, Befehle und Struktur beschreiben. Dieser Ausschnitt kann veraltet oder unvollständig sein und wird im Prompt entsprechend nur als Zusatzkontext behandelt. Für produktiven Betrieb wären Datenklassifikation, Secret-Erkennung und eine projektabhängige Freigabe vor externer Modellübertragung erforderlich.

Auch die lokalen Artefakte erweitern die Schutzfläche. `context.md`, `context.json` und Repository-Zusammenfassungen können Tickettext, technische Hinweise oder interne Pfade enthalten. Der Prototyp validiert die Pfadsegmente nicht und implementiert noch kein Rollen-, Aufbewahrungs- oder Löschkonzept. Nachvollziehbarkeit und Datenschutz stehen hier in einem Zielkonflikt: Persistenz erleichtert Prüfung und reproduzierbaren Publish, erzeugt aber eine zusätzliche schützenswerte Kopie.

5.9. Testbarkeit und verbleibende Grenzen

Der Mock-Provider und kleine Parser-, Schema- und Formatterfunktionen ermöglichen lokale Tests ohne reale Modellkosten. Processor und Publisher erzeugen ihre GitLab-beziehungsweise AI-Dienste jedoch als Modulkonstanten und besitzen keine durchgängige Dependency Injection. Die Tests prüfen daher vor allem Teilfunktionen; Zeitstempelprüfung, Routenautorisierung, Pfadbegrenzung, Publish-Reihenfolge und orchestrierte GitLab-Abläufe bleiben ungetestet.

Die Teststrategie unterscheidet reine Funktionen, Komponenten und HTTP-Routen. Parser, Schema und Formatter werden mit festen Ein- und Ausgaben geprüft. Für GitLab und OpenCode kommen Testdoubles beziehungsweise temporäre ausführbare Programme zum Einsatz. Der Webhook-Routentest verbindet Middleware und Router ohne produktive GitLab-Projekte oder Modellkosten. Der vollständige Process-/Publish-Pfad mit Dateisystem und realer GitLab-Schnittstelle wird nicht als Ende-zu-Ende-Fall ausgeführt.

Diese Isolation erleichtert Wiederholungen, kann jedoch falsche Annahmen über externe Verträge konservieren. Die Tests stabilisieren das implementierte `timestamp:body`-Verfahren, prüfen aber weder die aktuellen Signaturvorgaben von GitLab noch die Aktualität des Zeitstempels. Erforderlich bleiben reale Schnittstellen-, OpenCode-Smoke- und vollständige Process-/Publish-Tests sowie eine inhaltliche Bewertung an einem Ticketkorpus.

Die Implementierung belegt den in Tabelle 5.1 genannten Kern: Modelloutput wird strukturell begrenzt, Analyse und Publish sind getrennt. Die offenen Muss-Anforderungen an Aktivierung, Authentifizierung, Pfade und Publish-Reihenfolge sind Gegenstand der Evaluation in Kapitel 6.

6. Qualitätssicherung und Evaluation

Dieses Kapitel beschreibt die Evaluationsmethode und berichtet die dabei erhobenen Befunde. Bewertet wird der Repository-Stand des Dev-Assist-Prototyps gegen die Anforderungen aus Kapitel 3. Ein erfolgreicher Build oder bestandener Test gilt dabei nur als Evidenz für den konkret geprüften Fall. Er belegt weder einen kontrollierten Gesamtworkflow noch die fachliche Qualität realer Modellantworten.

Die Untersuchung ist als verifikationsorientierte Evaluation eines einzelnen Prototyps angelegt. Sie verbindet einen dokumentierten Build- und Testlauf, eine statische Prüfung sicherheits- und integritätsrelevanter Kontrollflüsse sowie den Vergleich mit externen GitLab-Verträgen. Es handelt sich nicht um ein kontrolliertes Experiment und nicht um eine Feldstudie. Die Methode wird daher gemeinsam mit ihren Aussagegrenzen dargestellt (vgl. Easterbrook et al., 2008, S. 285 f. und S. 302 f.).

6.1. Evaluationsfragen und Analyseinheit

Die beiden Forschungsfragen der Arbeit betreffen den Informationsbedarf von Tickets und die kontrollierte technische Integration. Für ihre technische Prüfung werden vier engere Evaluationsfragen verwendet. Sie dienen nicht als zusätzliche Forschungsfragen, sondern operationalisieren, welche Aussage aus welcher Evidenz abgeleitet werden darf:

1. **EF-1 – Reproduzierbarkeit:** Lässt sich der fixierte Repository-Stand mit den dokumentierten Werkzeugen bauen, und liefern die vorhandenen automatisierten Tests einen erfolgreichen, wiederholbaren Lauf?
2. **EF-2 – funktionale Übereinstimmung:** Welche fachlichen Schritte von Aktivierung, Kontextaufbereitung, Analyse, Klärung, Vorschau, Persistenz und Publish sind im Quellcode vorhanden und durch passende Prüfungen belegt?
3. **EF-3 – Sicherheits- und Integritätsgrenzen:** Entsprechen Authentifizierung, Eingabevalidierung, Zustandskopplung, Berechtigungen und Reihenfolge produktiver Seiteneffekte den Muss-Anforderungen und den verwendeten externen

Verträgen?

4. **EF-4 – Nachweis der Modellwirkung:** Welche Aussagen über Faktentreue, Vollständigkeit, Qualität der Rückfragen und praktische Nutzbarkeit erlauben die tatsächlich erhobenen Daten?

Analyseeinheit ist jeweils ein überprüfbarer Übergang im Kernprozess, nicht eine einzelne Datei. Für die Ereignisannahme umfasst eine Einheit beispielsweise Eingangsheader, Rohkörper, Authentifizierungsentscheidung, Parsergebnis und Relevanzprüfung. Für Publish umfasst sie gespeicherten Kontext, sichtbaren Vorschlag, Freigabeentscheidung, Issue-Änderung und das Löschen der Bot-Kommentare. Diese Wahl verhindert, dass eine isoliert korrekt arbeitende Funktion stellvertretend für einen unvollständigen Gesamtübergang bewertet wird.

Die Evaluationsfragen verbinden drei Ebenen. Auf der *Artefaktebene* wird festgestellt, welche Module, Konfigurationen und Tests vorliegen. Auf der *Verhaltensebene* wird geprüft, welche Entscheidung ein ausführbarer oder statisch verfolgter Pfad für konkrete Eingaben trifft. Auf der *Aussageebene* wird begrenzt, welche Schlussfolgerung daraus zulässig ist. Ein bestandener Parser-Test kann damit die Normalisierung einer Payload belegen, aber weder die Authentizität einer realen Lieferung noch die fachliche Qualität des späteren Vorschlags. Diese explizite Schlusskette bildet den methodischen roten Faden des Kapitels.

6.2. Untersuchungsstand und Artefakte

Die Prüfung wurde am 31.07.2026 unter Windows mit Node.js v24.11.0 und npm 11.6.1 durchgeführt. Der untersuchte Programmstand ist durch den Git-Commit d179eb22beb0a59006869a3acc7829f3070fc678 identifiziert. Die Dokumentationsänderungen dieser Arbeit betreffen nicht den geprüften Quellcode. Als ausführbare Prüfbasis dienten `npm run build` und `npm test`.

Tabelle 6.1: Ergebnisse des dokumentierten Build- und Testlaufs

Prüfgröße	Ergebnis
TypeScript-Build	Erfolgreich; keine TypeScript-Fehler
Automatisierte Tests	60 von 60 Tests in 15 Testsuiten bestanden
Teststatus und Laufzeit	Keine fehlgeschlagenen, übersprungenen oder offenen Tests; ca. 0,6 Sekunden
Testabdeckung	Nicht erhoben; Coverage-Skript und Coverage-Konfiguration fehlen
Kontinuierliche Integration	Keine GitHub- oder GitLab-CI-Konfiguration im Repository

Tabelle 6.1 dokumentiert einen einzelnen erfolgreichen Lauf. Die Tests prüfen vor allem deterministische Parser-, Schema-, Formatter-, Laufzeit- und Aufbereitungslogik. Ein positiver Ende-zu-Ende-Test vom realen GitLab-Webhook über Modellaufruf und Persistenz bis zur Issue-Änderung ist nicht vorhanden. Die Testzahl darf deshalb weder als Coverage-Angabe noch als Zuverlässigkeitskennzahl interpretiert werden.

Anstelle einer langen Dateiliste werden die projektinternen Quellen als versionierter Projektstand geführt und in Tabelle 6.2 nach ihrer Funktion gruppiert (vgl. Werner, 2026). Dadurch bleiben Version, Standdatum und Bezug der Evidenz nachvollziehbar.

Tabelle 6.2: Untersuchte projektinterne Artefaktgruppen

Artefaktgruppe	Umfang	Evaluationszweck
Implementierung	src/, opencode.json	Kontrollflüsse, Daten- und Vertrauensgrenzen
Testbasis	15 Dateien unter tests/	Ausführungsevidenz für festgelegte Fälle
Build und Konfiguration	package.json, tsconfig.json, .env.example	Befehle, Versionen und Sicherheitsstandardwerte
Ziel- und Betriebsbeschreibung	README.md, descriptions/project_description.txt, AGENTS.md	Sollprozess, Nutzung und Projektkontext

6.3. Prüfprotokoll und Nachverfolgbarkeit

Die Prüfung folgt einer festen Reihenfolge. Zunächst wird der Untersuchungsstand durch Commit, Datum und Laufzeitumgebung eingefroren. Anschließend werden Build und vollständige Testsuite ohne selektives Auslassen einzelner Tests ausgeführt. Im dritten Schritt werden die Testorakel gelesen: Entscheidend ist nicht allein, dass ein Test grün ist, sondern welches Verhalten seine Assertions tatsächlich verlangen. Danach wird für jede Muss-Anforderung der relevante Kontrollfluss von der Eingabe bis zu allen beobachtbaren Seiteneffekten verfolgt. Externe Annahmen, insbesondere die Signaturvorgaben von GitLab, werden zuletzt mit der zum Prüfdatum geltenden Primärdokumentation verglichen.

Tabelle 6.3 macht dieses Vorgehen und die jeweilige Entscheidungsgrenze nachvollziehbar. Die Schritte sind komplementär. Eine reine Quellcodelektüre kann einen beabsichtigten Zweig erkennen, aber nicht seine Ausführbarkeit bestätigen. Ein Komponententest kann ein Verhalten ausführen, aber einen unzutreffenden externen Vertrag als Testorakel festschreiben. Erst der Abgleich der Evidenzarten erlaubt eine begrenzte Konformitätsaussage.

Tabelle 6.3: Prüfprotokoll und zulässige Schlussfolgerungen

Prüfschritt	Erhobener Beleg	Entscheidungsgrenze
Stand fixieren	Commit, Standdatum, Plattform und Werkzeugversionen	Spätere Dokumentänderungen dürfen nicht als geprüfte Implementierung erscheinen
Build ausführen	Erfolgreiche TypeScript-Kompilation oder konkreter Fehler	Belegt statische Übersetzbarkeit, nicht Laufzeit- oder Fachverhalten
Tests ausführen und Orakel lesen	Teststatus, Suite-Zuordnung, Eingaben und Assertions	Belegt nur die ausgeführten Fälle und das im Test erwartete Verhalten
Kontrollfluss verfolgen	Guards, Fallbacks, Persistenz und Reihenfolge der Seiteneffekte	Belegt vorhandene Pfade; nicht erreichbare externe Zustände werden nicht simuliert
Vorgaben abgleichen	Sollanforderung und externe Primärdokumentation	Interne Konsistenz gilt nur bei Übereinstimmung mit den maßgeblichen Vorgaben als Konformität
Nachweislücke protokollieren	Fehlender Test, fehlende Messung oder fehlende Betriebsdaten	Abwesenheit von Evidenz wird als <i>nicht untersucht</i> , nicht automatisch als Fehlfunktion bewertet

Für die Nachverfolgbarkeit erhält jeder Befund vier Bestandteile: Anforderungsbezug, beobachteter Ist-Pfad, zugehörige Evidenz und Statusbegründung. Diese Form ist besonders bei negativen Ergebnissen wichtig. Wird eine Signaturprüfung als nicht erfüllt bewertet, genügt weder der Hinweis auf eine fehlende Testdatei noch eine allgemeine Sicherheitsannahme. Erforderlich ist die konkrete Gegenüberstellung aus gefordertem Nachrichtenformat, implementierter Berechnung, Standardwerten und Behandlung fehlender Schlüssel. Entsprechend wird die Publish-Reihenfolge anhand der beobachtbaren Aufrufreihenfolge und ihrer Fehlerwirkung beurteilt, nicht anhand des Funktionsnamens oder der beabsichtigten Architektur.

Die Tests wurden nicht nachträglich verändert, um sie an die Zielanforderungen anzupassen. Ihre unveränderte Ausführung beantwortet EF-1 und liefert für EF-2 Ausführungsevidenz. Abweichungen zwischen einem grünen Test und der Sollanforderung werden als eigener Befund behandelt. Dieses Vorgehen vermeidet eine zirkuläre Argumentation, in der die Implementierung über selbst gesetzte Erwartungen ihre eigene

Konformität bestätigt.

6.4. Bewertungslogik

Die Prüfgegenstände folgen den Übergängen, an denen Daten ihre Vertrauens- oder Darstellungsgrenze wechseln: vom Webhook zum internen Ereignis, vom Ticketkontext zum Modellprompt, von der Modellantwort zum validierten Objekt und vom gespeicherten Vorschlag zur Issue-Änderung in GitLab. Für jeden Übergang werden drei Evidenzarten unterschieden. *Ausführungsevidenz* bestätigt einen mit festen Eingaben ausgeführten Fall. *Implementierungsevidenz* verfolgt den vorhandenen Kontrollfluss statisch. *Vorgabenevidenz* vergleicht Implementierung und Testannahme mit der fachlichen Anforderung oder externen Schnittstelle.

Eine Anforderung wird als *nachgewiesen* bewertet, wenn Ziel- und Ist-Verhalten übereinstimmen und passende Evidenz vorliegt. *Teilweise nachgewiesen* bezeichnet vorhandene Bausteine bei offenem Gesamtpfad oder unvollständigem Zielverhalten. *Nicht erfüllt* gilt, wenn die Implementierung der Muss-Anforderung widerspricht, auch wenn ein Test genau dieses abweichende Verhalten bestätigt. *Nicht untersucht* wird für fachliche Wirkung oder Betriebsbedingungen ohne geeignete Daten verwendet. Diese Kategorien verhindern, dass interne Konsistenz mit externer Konformität verwechselt wird.

Sicherheits- und Änderungsgrenzen werden höher gewichtet als reine Darstellungsfragen. Eine falsche Überschrift ist korrigierbar; eine unberechtigte Verarbeitung oder vorzeitig gelöschte Diskussion kann dagegen Integrität und Nachvollziehbarkeit beeinträchtigen. Die Bewertung folgt deshalb der Kette Anforderung, Implementierung, Evidenz und begrenzte Interpretation.

6.4.1. Prüfung von Sicherheits- und Integritätsinvarianten

Für EF-3 reicht es nicht aus, einzelne Schutzfunktionen zu identifizieren. Eine Sicherheitsgrenze ist nur wirksam, wenn ihre Vorbedingungen auf jedem erreichbaren Eingang gelten, ein Fehler geschlossen behandelt wird und nachgelagerte Seiteneffekte die geprüfte Entscheidung nicht umgehen. Die Analyse formuliert deshalb drei Invarianten für den Kernprozess:

Ereignisinvariante: $verarbeiten(e) \Rightarrow authentifiziert(e) \wedge frisch(e) \wedge relevant(e)$

Freigabeinvariante: $veröffentlichen(r) \Rightarrow vollständig\ sichtbar(r) \wedge explizit\ freigegeben(r)$

Änderungsinvariante: $Kommentare\ löschen \Rightarrow Issue\text{-Aktualisierung\ erfolgreich}$

Die Ereignisinvariante koppelt Herkunft, Aktualität und fachliche Relevanz. Eine korrekte HMAC-Berechnung allein genügt nicht, wenn die Anwendung alte Lieferungen unbegrenzt akzeptiert. Umgekehrt ersetzt eine präzise Mention-Erkennung keine Authentifizierung. Für manuelle Routen ist ein äquivalenter Zugangsschutz erforderlich, auch wenn dort kein GitLab-Webhook vorliegt. Die Prüfung betrachtet daher alle Eingänge, die denselben Processor oder Publisher erreichen können.

Die Freigabeinvariante bindet die menschliche Entscheidung an eine konkrete Revision. Zeitliche Trennung zwischen Analyse und Publish ist dafür notwendig, aber nicht hinreichend. Ohne Revisionskennung kann ein gespeicherter Inhalt nach der Sichtprüfung verändert oder bereits während einer Klärung umfangreicher als der sichtbare Kommentar sein. Für die Bewertung wird deshalb nicht nur geprüft, ob ein Publish-Kommando existiert, sondern ob der Publisher Sichtbarkeit, Identität und Freigabestatus derselben Version nachweist.

Die Änderungsinvariante ordnet irreversible beziehungsweise schwer rekonstruierbare Seiteneffekte. Die Issue-Aktualisierung ist die fachliche Kernänderung; die Entfernung von Assistenzkommentaren ist nachgeordnet. Wird diese Reihenfolge umgekehrt, kann ein Fehler einen Zustand erzeugen, in dem weder das alte Ticket mit vollständiger Diskussion noch der neue Ticketstand verfügbar ist. Die sichere Reihenfolge muss aus dem Kontrollfluss hervorgehen und durch einen negativen Test mit fehlschlagendem Issue-Update abgesichert werden.

Neben diesen drei Kerninvarianten werden Eingabedomänen und Idempotenz geprüft. Projekt- und Issue-Kennungen sind nicht nur API-Parameter, sondern Bestandteile lokaler Pfade; ihre zulässige Domäne muss deshalb vor der Pfadbildung feststehen. Deduplizierung benötigt eine pro Lieferung stabile Identität und für mehrere Instanzen einen gemeinsamen Zustand. Ein flüchtiger Schlüssel kann kurzfristige Wiederholungen begrenzen, erfüllt aber keine allgemeine Exactly-once-Garantie. Die Bewertung vermeidet eine solche stärkere Aussage.

6.4.2. Trennung technischer und semantischer Evaluation

EF-4 verlangt eine andere Evidenz als EF-1 bis EF-3. Die Schemavalidierung misst, ob eine Antwort die erwarteten Felder und Datentypen besitzt. Sie misst nicht, ob ein vorgeschlagenes Akzeptanzkriterium aus dem Ticketkontext folgt oder ob eine Rückfrage die wichtigste Informationslücke adressiert. Entsprechend werden drei Qualitätsdimensionen getrennt: *strukturelle Qualität* als maschinelle Verarbeitbarkeit, *semantische Qualität* als Faktentreue und Kontextdeckung sowie *pragmatische Qualität* als Nutzen für Umsetzung und Prüfung. Diese Trennung folgt dem in Kapitel 2 hergeleiteten Qualitätsverständnis und der Unterscheidung zwischen Artefakteigenschaft, Bewertung und Wirkung bei Frattini et al. (2023, S. 507–520).

Im vorhandenen Repository lassen sich nur strukturelle Eigenschaften automatisiert prüfen. Für eine semantische Messung wären unabhängige Referenzinformationen oder fachkundige Urteile erforderlich. Ein valides Untersuchungsobjekt wäre nicht die einzelne Modellantwort ohne Kontext, sondern das Tripel aus Ausgangsticket, verfügbarer Evidenz und erzeugtem Entwurf. Bewertende müssten für jede neue Aussage kennzeichnen, ob sie belegt, widersprüchlich, unentscheidbar oder frei ergänzt ist. Rückfragen wären getrennt nach Relevanz, Kritikalität und Beantwortbarkeit zu beurteilen. Mehrere Läufe derselben Konfiguration wären notwendig, um stochastische Streuung nicht mit Unterschieden zwischen Tickets zu verwechseln.

Auch pragmatischer Nutzen benötigt Vergleichsdaten. Bearbeitungszeit, Zahl der Klärungsschleifen und Korrekturaufwand wären gemeinsam mit Faktentreue und Prüfbarkeit zu erheben. Eine schnelle Übernahme eines sachlich falschen Entwurfs wäre kein positiver Effekt. Da weder ein solches Korpus noch unabhängige Bewertungen oder Vergleichsbedingungen vorliegen, lautet das Ergebnis für EF-4 *nicht untersucht*. Diese Entscheidung ist keine Bewertung gegen das Modell, sondern eine methodische Grenze des verfügbaren Materials.

6.5. Befunde entlang der Prozessgrenzen

6.5.1. Ereignisannahme und Zugriffsschutz

FA-1 fordert `@dev-assist` am Anfang der ersten inhaltlichen Zeile. Die Implementierung akzeptiert dagegen Titel- und Inline-Mentions; FA-1 ist nicht erfüllt. Issue- und Note-Ereignisse werden vereinheitlicht, das normalisierte Ereignis enthält jedoch keinen Autor. FA-2 ist deshalb nur teilweise erfüllt. Der Deduplizierungsschlüssel wird vor der fachlichen Relevanzprüfung gesetzt und verwendet Projekt, Issue, Ereignisart sowie eine Payload-ID oder `no-id`. Unterschiedliche legitime Lieferungen können innerhalb der TTL kollidieren; Neustarts und mehrere Instanzen werden nicht abgedeckt.

Die Webhook-Prüfung verwendet `x-gitlab-signature`, `x-gitlab-timestamp` und eine HMAC über `timestamp:body`; eine Zeitstempelprüfung fehlt. Ist kein Secret konfiguriert, meldet der Verifier selbst bei aktivierter Signaturpflicht Erfolg. Nach den aktuellen Signaturvorgaben von GitLab werden dagegen `webhook-id`, `webhook-timestamp` und `webhook-signature` verwendet; signiert wird `{message_id}.{timestamp}.{body}`, und der Zeitstempel muss aktuell sein (vgl. GitLab Documentation, o. J.a, Abschnitt "Signing tokens"). Die Tests bestätigen damit nur das implementierte Verfahren, nicht die externen Vorgaben. NFA-6 ist nicht erfüllt.

Auch die manuellen Process- und Publish-Routen besitzen keine eigene Bearer-Authentifizierung. Projekt- und Issue-IDs werden nicht durchgängig als positive Ganzzahlen validiert; aus ihnen gebildete Speicherpfade besitzen deshalb keine hinreichend belegte Eingabegrenze. Diese Punkte betreffen die Zugangskontrolle des Kernprozesses und können nicht durch eine spätere Schemavalidierung kompensiert werden.

6.5.2. Modellgrenze und Ausgabequalität

Die Anwendung akzeptiert nur ein JSON-Objekt mit den sechs Feldern `title`, `description`, `acceptanceCriteria`, `technicalContext`, `proposedSolution` und `openQuestions`. Falsche Typen und unbekannte Felder werden abgelehnt. Damit ist die strukturelle Antwortgrenze für die geprüften Fälle nachgewiesen. Das fachliche Zielmodell aus Kapitel 3 enthält darüber hinaus Quellenbasis, Risiken und Validie-

runingsschritte; diese Kategorien sind im kompakten Ausgabeschema nicht eigenständig repräsentiert.

Die OpenCode-Auswertung akzeptiert den ersten schemafähigen Kandidaten in Fundreihenfolge und schließt bekannte Promptbeispiele nicht ausdrücklich aus. Der Formatter markiert leere Arrays, lässt aber inhaltliche Platzhalter wie `unknown` unverändert. Ein formal gültiger Output kann daher weiterhin der falsche Kandidat oder fachlich unzureichend sein. Reale Modellqualität, Versionskompatibilität, wiederholte Läufe und semantische Verarbeitung von Anhängen wurden nicht untersucht.

6.5.3. Persistenz und Publish

Im Klärungsfall zeigt GitLab nur Rückfragen, während der Processor den vollständigen Analysekontext persistiert. Der Publisher prüft weder, ob zuvor ein Vollvorschlag sichtbar war, noch ob offene Fragen einer Freigabe entgegenstehen. Die geforderte Invariante, nur den vollständig sichtbaren und freigegebenen Stand zu veröffentlichen, ist damit nicht abgesichert.

Zusätzlich löscht der Publisher ausgewählte Dev-Assist-Kommentare vor `updateIssue`. Schlägt die anschließende Issue-Aktualisierung fehl, kann bereits Diskussionskontext verloren sein. FA-11 ist deshalb nicht erfüllt. Die in Kapitel 5 dargestellte Zustandslücke ist keine bloße Komfortfrage, sondern betrifft die Integrität des Freigabeprozesses.

6.6. Überleitung zu den Ergebnissen

EF-1 ist für den dokumentierten Lauf beantwortet. EF-2 bleibt auf ausgewählte Teilpfade begrenzt. EF-3 weist Widersprüche zu Muss-Anforderungen und zu den Signaturvorgaben von GitLab nach. EF-4 ist nicht untersucht. Die konsolidierte Statusmatrix folgt in Kapitel 7; Ursachen, Alternativerklärungen und Validität in Kapitel 8.

7. Ergebnisse

Dieses Kapitel verdichtet die in Kapitel 6 erhobenen Einzelbefunde zu einem Gesamtbild. Es wiederholt weder die dort dokumentierten Prüfpfade noch die zugrunde liegenden Implementierungsdetails, sondern beantwortet drei Ergebnisfragen: Welche Anforderungen sind im untersuchten Stand nachweisbar erfüllt, an welchen Prozessgrenzen bestehen wesentliche Abweichungen und welche Reichweite besitzt der erbrachte Nachweis? Die Interpretation der Ursachen und Folgen erfolgt erst in Kapitel 8; die abschließende Beantwortung der Forschungsfragen und die Priorisierung weiterer Arbeiten folgen in Kapitel 9.

7.1. Konsolidierter Ergebnisstand

Tabelle 7.1 fasst die entscheidungsrelevanten Ergebnisse zusammen. Der Status *teilweise erfüllt* bezeichnet eine vorhandene Teilfunktion, deren geforderter Umfang oder Nachweis begrenzt bleibt. *Nicht erfüllt* bedeutet, dass das beobachtete Verhalten der Anforderung widerspricht oder eine notwendige Kontrollgrenze fehlt. *Nicht untersucht* kennzeichnet dagegen keine nachgewiesene Fehlfunktion, sondern eine außerhalb des Evaluationsdesigns liegende Wirkungsaussage. Diese Unterscheidung ist insbesondere beim Modelloutput erforderlich: Seine formale Verarbeitung wurde geprüft, seine fachliche Qualität jedoch nicht empirisch bewertet.

Tabelle 7.1: Konsolidierter Anforderungs- und Nachweisstand des untersuchten Prototyps (eigene Darstellung)

Anforderung oder Grenze	Ergebnis	Verdichtete Aussage
FA-1: Aktivierung	Nicht erfüllt	Titel- und Inline-Mentions aktivieren den Dienst entgegen der Beschränkung auf den Anfang der ersten inhaltlichen Zeile.
FA-2: Autorinformation	Teilweise erfüllt	Das normalisierte Ereignis enthält nicht die geforderte Autorinformation.
NFA-6: Authentifizierung	Nicht erfüllt	Die Prüfung ist optional, weicht von den Signaturvorgaben von GitLab ab und enthält keine Zeitstempelprüfung.
Manuelle Routen, Kennungen und Pfade	Nicht erfüllt	Zugriffsschutz und hinreichende Validierung von IDs und daraus abgeleiteten Speicherpfaden fehlen.
Abgleich von Vorschau und gespeichertem Stand	Nicht erfüllt	Der gespeicherte Publish-Stand kann über den vollständig sichtbaren und bestätigten Vorschlag hinausgehen.
FA-11: Löschreihenfolge	Nicht erfüllt	Kommentare können vor einer erfolgreich abgeschlossenen Issue-Aktualisierung gelöscht werden.
NFA-7: Deduplizierung	Teilweise erfüllt	Filterung ist vorhanden, bleibt aber prozesslokal und von potenziell mehrdeutigen Payload-Feldern abhängig.
FA-4 bis FA-8: Modelloutput	Formal teilweise; semantisch nicht untersucht	Ausgabeschema und Verarbeitung sind für ausgewählte Fälle geprüft; fachliche Richtigkeit und Nutzbarkeit nicht.
Kontrollierter Gesamtworkflow	Nicht nachgewiesen	Teilfunktionen sind belegt; offene Muss-Anforderungen sowie fehlende Ende-zu-Ende- und Wirkungsnachweise verhindern den Gesamtnachweis.

Die Übersicht ist asymmetrisch: Die Komponentenkette von Ereignisannahme, Kontext, validierter Analyse, Rückfrage oder Vorschlag und getrenntem Publish ist vorhanden. Mehrere Abweichungen liegen jedoch an Aktivierung, Authentizität, manuellem Zugriff und der Bindung zwischen Vorschau und Veröffentlichung. Bestandene Tests belegen

die implementierte Verarbeitung, nicht automatisch die Sollanforderung und nicht die fachliche Qualität der Modellantworten.

7.2. Illustrierter Ablauf

Damit die Prozessgrenzen nicht nur als Statusmatrix erscheinen, verdichtet dieser Abschnitt den Kernablauf an einem konstruierten Beispiel. Es handelt sich nicht um betriebliche Originaldaten und nicht um einen empirischen Evaluationsfall. Der Verlauf folgt dem Ausgabeschema und der Formatter-Logik des untersuchten Stands; ein ausführlicherer Ticketverlauf steht in Anhang A.

Ausgangslage ist ein unvollständiges Issue mit dem Titel “Login funktioniert nicht” und einer einzeiligen Beschreibung ohne Reproduktion, betroffene Rolle oder erwartetes Ergebnis. Nach Aktivierung durch `@dev-assist` lädt der Processor Issue und Notes, übergibt den begrenzten Kontext an den Analyseagenten und erhält ein schema-konformes JSON. Weil wesentliche Angaben fehlen, entstehen mindestens zwei substantielle offene Fragen, etwa nach dem beobachtbaren Fehlverhalten und dem erwarteten Ergebnis. GitLab zeigt in diesem Fall nur den Klärungskommentar; lokal wird dennoch der vollständige Analysekontext gespeichert.

Beantwortet eine Person die Fragen im Issue, kann eine erneute Analyse Ziel, Akzeptanzkriterien und technischen Kontext in die vorgesehenen Felder überführen. Sichtbar wird dann ein strukturierter Vorschlag. Erst `@dev-assist publish` würde diesen Stand in Titel und Beschreibung schreiben. Im untersuchten Stand prüft der Publisher jedoch weder, ob zuvor ein vollständiger Vorschlag sichtbar war, noch ob offene Fragen einer Freigabe entgegenstehen. Das Beispiel macht damit denselben Befund wie Tabelle 7.1 konkret: Die Komponentenkette ist vorhanden, die revisionsgebundene Kontrolle nicht.

7.3. Zustandslogik des Vorschlags

Für die Einordnung der Ergebnisse ist der Zustand eines Vorschlags wichtiger als die bloße Existenz einer Kontextdatei. Ein kontrollierter Ablauf unterscheidet mindestens die Zustände *neu*, *in Analyse*, *in Klärung*, *als vollständige Vorschau sichtbar*, *explizit freigegeben* sowie *veröffentlicht* oder *Fehler*. Ein Analyseergebnis mit mehreren offenen Fragen darf

demnach in den Klärungszustand übergehen, aber noch nicht als veröffentlichungs-fähiger Vorschlag gelten. Erst nachdem der vollständige Stand sichtbar gemacht und ausdrücklich freigegeben wurde, ist der Übergang zu Publish zulässig.

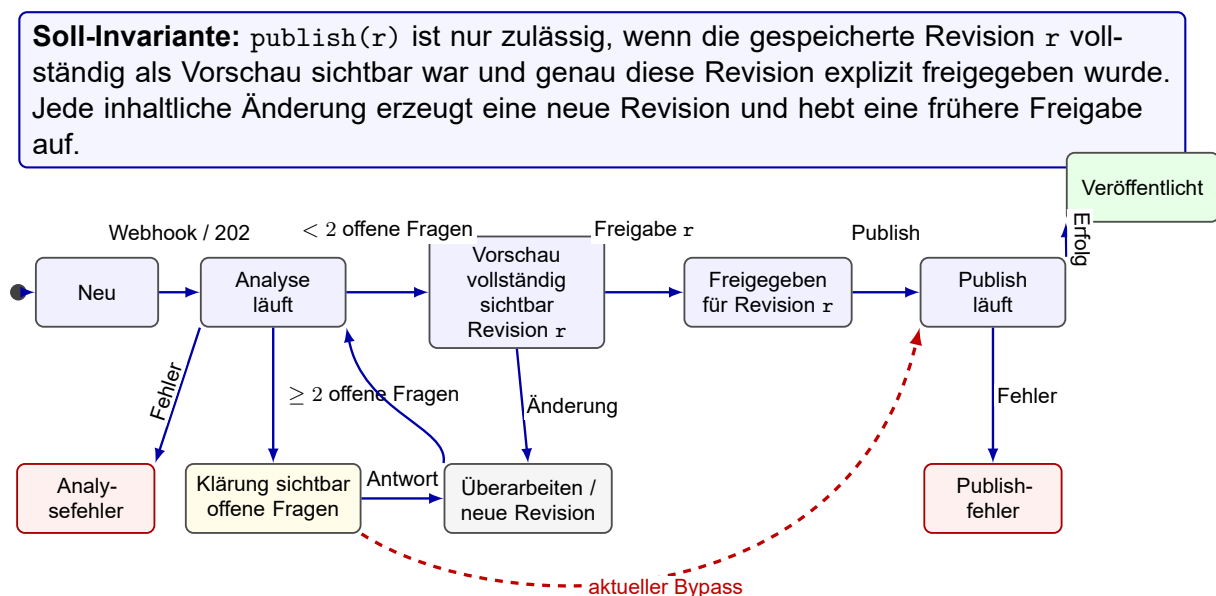
Die dafür erforderliche Soll-Guard lautet:

Publish-Guard:

persistierte Version = vollständig sichtbare Version

UND *explizit freigegeben*

Abbildung 7.1 überführt die Guard in ein Zustandsmodell. Blau dargestellte Übergänge bilden den erforderlichen kontrollierten Ablauf; der rot gestrichelte Pfad markiert die im untersuchten Stand mögliche Abkürzung.



Aktuelle Implementierungslücke: Der Processor persistiert `fullContext` auch bei einer sichtbaren Klärungsnachricht. Die Metadaten enthalten keinen Workflowstatus, keine Revisions-ID und keinen Freigabemarker. Deshalb kann `publish` den gespeicherten Vollvorschlag übernehmen, ohne Sichtbarkeit und Freigabe derselben Revision zu prüfen.

Abbildung 7.1.: Soll-Zustandsmodell eines Dev-Assist-Vorschlags und Abweichung der aktuellen Implementierung. Die Freigabe muss an dieselbe Revision gebunden sein, die vollständig in GitLab sichtbar war. Der rot gestrichelte Übergang zeigt die fehlende Status-, Revisions- und Freigabepfung. Eigene Darstellung auf Grundlage des untersuchten Quellcodes, Stand 31.07.2026.

Die Guard verbindet Sichtbarkeit, Versionstreue und menschliche Entscheidung. Erforderlich sind eine versionierte Vorschau, ein an dieselbe Revision gebundener Freigabe-

status und eine erneute Prüfung unmittelbar vor der Issue-Änderung. Der untersuchte Stand erzwingt diese Kopplung nicht: Während einer sichtbaren Klärung kann ein umfangreicherer Stand gespeichert und später ohne Statusprüfung übernommen werden. Die zeitliche Trennung von Analyse und Publish bildet daher noch keinen kontrollierten Human-in-the-Loop-Prozess.

Auch Fehler müssen in diesem Modell einen sicheren Zustand ergeben. Scheitert das Issue-Update, müssen Issue, Vorschau und Diskussion unverändert nachvollziehbar bleiben. Erst nach bestätigter Kernänderung darf das Löschen der Bot-Kommentare folgen. Damit ergänzen Revisionsprüfung und fehlersichere Reihenfolge einander.

7.4. Reichweite des Ergebnisses

Der Nachweis endet bei den untersuchten deterministischen Komponenten. Reale Ende-zu-Ende-Läufe, semantische Ticketqualität, Lastverhalten und Nutzung im Team wurden nicht geprüft; Zeitersparnis oder Fehlerreduktion lassen sich daher nicht ableiten. Der Prototyp ist ein technischer Proof of Concept ausgewählter Komponenten, aber kein Nachweis eines kontrollierten oder produktionsreifen Gesamtworkflows.

8. Diskussion

Kapitel 6 dokumentiert die Einzelevidenz, Tabelle 7.1 den verdichteten Ergebnisstand. Die Diskussion wiederholt diese Befunde nicht, sondern interpretiert ihren Beitrag, prüft Alternativerklärungen und begrenzt Gültigkeit und Übertragbarkeit. Die Forschungsfragen werden erst in Kapitel 9 abschließend beantwortet.

8.1. Interpretation des Ergebnisbeitrags

Dev-Assist setzt an einem empirisch bekannten Informationsproblem an. In technischen Tickets fehlen häufig Reproduktionsschritte, Systemausgaben, Testfälle oder entscheidungsrelevanter Kontext; wesentliche Angaben entstehen oft erst im Dialog (vgl. Bettenburg et al., 2008, S. 308 f.; Breu et al., 2010, S. 301 ff.; Chaparro et al., 2017, S. 396 f.). Der nachgewiesene Beitrag liegt in der Anwendungshülle: Kontextbündelung, festes Ausgabeschema und getrennte Issue-Änderung. Tabelle 7.1 begrenzt die Reichweite: Ohne gesicherte Ereignisherkunft, Freigabestand und Fehlerreihenfolge bleibt es ein Proof of Concept ausgewählter Komponenten, kein kontrollierter Gesamtprozess.

8.1.1. Formale Struktur und semantische Qualität

Das Ausgabeschema verbessert zunächst die maschinelle und visuelle Struktur. Ungültige Typen oder unbekannte Felder können zurückgewiesen werden. Nach Lucassen et al. sind syntaktische, semantische und pragmatische Qualität jedoch zu trennen (vgl. Lucassen et al., 2016, S. 383 f. und S. 386 f.). Ein wohlgeformtes Akzeptanzkriterium kann fachlich falsch, ein korrektes Detail irrelevant und ein vollständiger Abschnitt nur scheinbar vollständig sein.

Aktuelle empirische Forschung bestätigt diese Grenze. Bei Lubos et al. war die anfängliche Übereinstimmung zwischen menschlicher und LLM-basierter Qualitätsbewertung teilweise schwach, obwohl Erklärungen und Überarbeitungen häufig als hilfreich galten (vgl. Lubos et al., 2024, S. 389–397). Paiva et al. berichten hohe sprachliche Qualitätsbewertungen, aber begrenzte exakte Übereinstimmung zwischen menschlichen

Urteilen und einem LLM-Judge sowie teilweise spekulative Ergänzungen (vgl. Paiva et al., 2026, Abschnitte 3–5). Mahbub et al. zeigen zusätzlich begrenzte Präzision bei Ambiguität, Inkonsistenz und Unvollständigkeit, insbesondere bei fehlenden dokumentweiten Bezügen und Fachwissen (vgl. Mahbub et al., 2024, S. 171979–171989). Für Dev-Assist ist Schemaerfüllung damit eine notwendige Verarbeitungsbedingung, kein fachliches Qualitätsurteil.

8.1.2. Rückfragen und Unsicherheitssteuerung

Der Rückfragemodus übersetzt Unsicherheit in Kooperation. K C et al. zeigen an realen Feature Requests, dass aus Mehrdeutigkeit und Unvollständigkeit konkrete Klärungsfragen abgeleitet werden können; ihre Beantwortung und Einordnung verbleiben jedoch bei Menschen (vgl. K C et al., 2025, Abschnitte III–V). Die Anzahl offener Fragen ist dabei kein ausreichendes Kritikalitätsmaß. Eine einzelne Sicherheitsentscheidung kann blockierend sein, mehrere Details können nachgeordnet sein. Die Schwellenlogik des Prototyps ist daher nur eine Heuristik.

Für die Gestaltung folgt daraus, dass eine Rückfrage die zugrunde liegende Informationslücke und deren Entscheidungsauswirkung erkennen lassen sollte. Eine Antwort darf erst dann als geklärt gelten, wenn ihre Herkunft nachvollziehbar ist; andernfalls bleibt die Unsicherheit im Entwurf sichtbar. Menschliche Beteiligung bezeichnet hier fachliche Klärung und nicht bereits die spätere formale Freigabe.

8.2. Nutzenhypothesen, Aufwand und Alternativen

Plausibel ist eine Entlastung bei wiederkehrender Konsolidierung: Angaben werden geordnet, beantwortete Fragen zusammengeführt und im bestehenden GitLab-Arbeitsort in eine einheitliche Struktur überführt. Gemessen wurden weder Bearbeitungszeit noch Klärungsrunden, Akzeptanzrate, Korrekturaufwand oder Fehlerquote. Aus dem vorhandenen Material folgt keine Wirkungsbehauptung.

Tabelle 8.1 ordnet den Aufwand ohne isolierte Tokenpreise. Entscheidend sind Prüfzeit, Fehlerrate und Ticketmenge gemeinsam. Eine beobachtete Verbesserung könnte bereits durch ein verpflichtendes Template, einfachere Ausgangstickets oder einen Neu-

heitseffekt entstehen. Saleem et al. berichten bei mehreren Requirements-Engineering-Aufgaben bessere Ergebnisse spezialisierter Verfahren; generative Modelle waren insbesondere bei Fragebeantwortung konkurrenzfähig (vgl. Saleem et al., 2025, Abschnitte “Results” und “Conclusion”). Relevante Baselines sind daher manuelle Aufbereitung, deterministische Templates und spezialisierte Modelle.

Tabelle 8.1: Wirtschaftliche Einordnung der Aufbereitungsalternativen ohne Wirkungsnachweis

Alternative	Aufwand und Nutzen	Grenze
Issue-Template	Keine Modellkosten; einheitliche Felder ohne zusätzliche Oberfläche.	Keine dialogische Konsolidierung über Kommentare.
Manuelle Aufbereitung	Volle fachliche Kontrolle; Aufwand steigt mit verteiltem Kontext.	Wiederholte Rückfragen und uneinheitliche Struktur.
Dev-Assist (PoC)	Keine Extra-UI; Struktur und Rückfragen im Issue. Variabel: Modellaufruf, menschliche Prüfung, Webhook-Betrieb.	Semantische Güte ungemessen; Sicherheits- und Integritätslücken offen.

Mehr Kontext verbessert Ergebnisse nicht automatisch. ReqRAG erreichte in der industriellen Bewertung von Ibtasham et al. für rund 70 Prozent der Antworten positive Nützlichkeits- und Relevanzurteile, aber keine vollständige Übereinstimmung (vgl. Ibtasham et al., 2025, S. 277–292). Veraltete oder unpassende Fundstellen können eine Antwort verschlechtern. Eine Wirkungsevaluation müsste dieselben geschichteten Tickets unter mehreren Bedingungen bearbeiten, Varianten verdeckt beurteilen und Faktentreue sowie Aufwand messen. Qualitätssteigerung und Zeitgewinn bleiben offene Hypothesen.

8.3. Architektonische Zielkonflikte

Das feste Ausgabeschema schafft Beherrschbarkeit auf Kosten von Ausdrucksfähigkeit. Wenige typisierte Felder erleichtern Parsing und Rendering; Quellenbasis, Risiken und Annahmen bleiben jedoch in allgemeinen Textlisten verborgen. Zusätzliche Felder erhöhen wiederum Kopplung zwischen Prompt, Typen, Validator, Formatter und Publisher.

Das kompakte Ausgabeschema ist deshalb eine begrenzte technische Repräsentation, keine vollständige Ontologie der Ticketqualität.

HTTP 202 verkürzt die Webhook-Antwort, trennt aber Transportannahme und fachlichen Abschluss. Ohne dauerhaften Jobzustand sind Fehler und noch laufende Verarbeitung schwer unterscheidbar. Ähnlich stehen Übersichtlichkeit und Nachvollziehbarkeit in Spannung: Das Löschen von Bot-Kommentaren kann ein Issue bereinigen, zugleich aber Entscheidungs- und Fehlerhistorie entfernen; lokale Kontextdateien erhöhen Reproduzierbarkeit, erzeugen aber zusätzliche Datenkopien. Diese Konflikte erfordern explizite Zustände, Provenienz, Berechtigungen und Aufbewahrungsregeln. Höhere Modellqualität allein löst sie nicht.

8.4. Organisatorische Einbettung menschlicher Kontrolle

Ein Human-in-the-Loop ist nicht bereits durch ein Publish-Kommando verwirklicht. Kontrolle besitzt nur dann fachliche Bedeutung, wenn eine zuständige Person den vollständigen Vorschlag samt Quellen prüft. Flüssige, wiederholt plausible Ausgaben können sonst eine nur formale Bestätigung begünstigen. Die Revisionsbindung aus Kapitel 7 ist daher notwendig, aber nicht hinreichend.

Im Prozess sind fachliche Eingabe, assistierte Strukturierung und Verantwortung für den verbindlichen Stand zu unterscheiden. Besonders bei Sicherheits-, Datenschutz- oder Architekturannahmen muss erkennbar sein, wer welche Revision bestätigt hat. Autor, Zeitstempel und Revisions-ID sind damit Bestandteile der Verantwortungszuordnung. Zugleich sollte die Oberfläche direkte Belege, Repository-Ableitungen und offene Modellannahmen unterscheidbar darstellen. Die begrenzte Übereinstimmung menschlicher und automatischer Qualitätsurteile spricht gegen einen bloß globalen Warnhinweis (vgl. Lubos et al., 2024, S. 389–397; Paiva et al., 2026, Abschnitte 3–5).

Prompt, Schema, Modell und Kontextauswahl verändern das System auch ohne sichtbare Oberflächenänderung. Sie benötigen Versionierung, dokumentierte Freigabe und Regressionstests, damit Abweichungen einer Konfiguration zugeordnet werden können. Technischer Audit-Trail und organisatorische Verantwortung müssen dabei denselben Revisionsstand beschreiben.

8.5. Gültigkeit der Untersuchung

Tabelle 8.2 fasst die wesentlichen Bedrohungen zusammen. Sie bewertet nicht das System, sondern begrenzt die Interpretation der Evidenz.

Tabelle 8.2: Zentrale Bedrohungen der Gültigkeit und ihre Konsequenzen

Dimension	Bedrohung	Konsequenz für die Interpretation
Konstruktvalidität	Build, Testzahl und Schemaerfüllung messen technische Teilfunktionen, nicht Ticketqualität oder Prozessnutzen.	Kein semantischer Qualitäts-, Coverage- oder Produktivitätsnachweis
Interne Validität	Test und Implementierung können dieselbe falsche Annahme über die Schnittstelle enthalten.	Interne Konsistenz bestätigt keine externe Konformität
Externe Validität	Einzelner Repository-Stand; keine realen Teams, Ticketkorpora, Last- oder Modellvarianten	Keine Erfolgsquoten oder statistische Generalisierung auf andere Projekte
Reliabilität	Dokumentierter technischer Lauf, aber keine wiederholte stochastische Modellevaluation	Technischer Stand reproduzierbar; Varianz realer Modellantworten unbekannt

Die stärkste Konstruktbedrohung wäre die Gleichsetzung bestandener Softwaretests mit Ticketqualität. Die Kategorie *nicht untersucht* verhindert, fehlende Wirkungsevidenz umzudeuten. Ausführung, Quellcode und Schnittstellenvorgaben wurden trianguliert; das ersetzt keine unabhängige Zweitprüfung des gesamten Abgleichs. Ticketart, Domäne, Sprache sowie Modell- und GitLab-Version können Ergebnisse verändern. Die dokumentierte Umgebung erhöht die Reproduzierbarkeit der technischen Bestandsaufnahme, nicht die Stabilität der probabilistischen Komponente.

8.6. Risiken und verantwortbarer Einsatz

Strukturelle Validierung verhindert weder erfundene Aussagen noch kritische Auslassungen. NIST fasst überzeugend präsentierte Falschaussagen als Confabulation beziehungsweise Hallucination (vgl. National Institute of Standards and Technology, 2024, S. 4 und S. 6). Quellenhinweise je Aussage, sichtbare offene Punkte und ein Abgleich zwischen Eingangskontext und Vorschlag sind deshalb aussagekräftiger als eine einzige Vollständigkeitskennzahl.

Tickettexte, Kommentare und Repository-Dateien sind nicht vertrauenswürdige Daten. Prompt Injection kann die Analyse verzerren, obwohl das Modell keine direkten GitLab-Rechte besitzt (vgl. Liu et al., 2023, S. 1; OWASP, 2025a, Abschnitt “LLM01:2025 Prompt Injection”). Der getrennte Publish-Schritt begrenzt übermäßige Handlungsfähigkeit, schützt aber nur bei autorisierter Freigabe derselben unveränderten Revision (vgl. OWASP, 2025b, Abschnitt “LLM06:2025 Excessive Agency”). Schema und Berechtigungstrennung reduzieren somit Schadenswirkung, nicht die Notwendigkeit menschlicher Inhaltsprüfung.

Issues, Logs und Quellcode können personenbezogene Angaben, interne URLs, Schwachstellen oder Secrets enthalten. Zusätzlicher Kontext erhöht Datenmenge und Angriffsfläche. Vor externer Modellübertragung sind Datenminimierung, Secret-Erkennung und Zweckbindung erforderlich; lokale Kontextdateien benötigen Zugriffsrechte und Löschfristen. Die Oberfläche sollte außerdem zwischen belegten, abgeleiteten und offenen Angaben unterscheiden, damit flüssige Sprache keine unangemessene Sicherheit vermittelt.

8.7. Übertragbarkeit

Webhook-Formate, APIs und Publish-Semantik sind GitLab-spezifisch. Übertragbar ist das Muster aus Ereignisfilter, begrenztem Kontext, validiertem Modelloutput, sichtbarer Vorschau und expliziter Freigabe. In einem anderen Issue-System müssen Authentifizierung, Rollen, Idempotenz und Ausgabeschema neu geprüft werden; eine Portierung der Komponenten überträgt keine nachgewiesene Prozesseigenschaft.

Auch fachlich bleibt das Ausgabeschema eine Mindeststruktur. Bug Reports, Feature-

Tickets sowie Wartungs- und Sicherheitsaufgaben benötigen unterschiedliche Zusatzinformationen. Sprache, Domäne, Modellversion, Prompt und Retrieval verändern zudem die Modellleistung. Übertragbar ist daher das Gestaltungsprinzip deterministischer Vertrauensgrenzen und menschlicher Verantwortung, nicht eine in dieser Arbeit ohnehin nicht erhobene Erfolgsquote.

8.8. Diskussionsfazit

Die Diskussion ordnet Dev-Assist als soziotechnische Assistenz ein: Formale Verarbeitbarkeit, fachliche Bewertung und organisatorische Verantwortung benötigen jeweils eigene Kontrollen und Evidenz. Kapitel 9 beantwortet auf dieser Grundlage die Forschungsfragen und priorisiert den weiteren Bedarf.

9. Fazit und Ausblick

Die Arbeit untersuchte den Informationsbedarf umsetzbarer GitLab-Tickets und die kontrollierte Einbindung eines KI-Assistenten. Dazu wurden ein fachliches Zielbild, der TypeScript-/Express-Prototyp Dev-Assist und ein systematischer Abgleich mit Anforderungen, Schnittstellenverträgen und Tests erarbeitet. Abschließend werden beide Forschungsfragen beantwortet und die nächsten Schritte priorisiert.

9.1. Beantwortung der Forschungsfragen

9.1.1. Erforderliche Informationen eines Tickets

Die erste Forschungsfrage lautet: *Welche Informationen muss ein GitLab-Ticket enthalten, damit Entwicklerinnen und Entwickler es nachvollziehbar umsetzen und überprüfen können?* Die Untersuchung ergibt sechs miteinander verbundene Informationsarten:

1. **Gegenstand und Ziel:** Titel und Zielbeschreibung benennen Änderung oder Störung und den angestrebten Nutzen.
2. **Kontext und Abgrenzung:** Rollen, Systeme, Voraussetzungen, Randbedingungen und Nicht-Ziele grenzen den Auftrag ein.
3. **Erwartetes Verhalten:** Soll- und Ist-Zustand müssen unterscheidbar sein; Bug Reports benötigen insbesondere Reproduktionsschritte.
4. **Prüfkriterien:** Beobachtbare Akzeptanzkriterien machen das Ergebnis überprüfbar.
5. **Technische Evidenz:** Logs, Fehlermeldungen und Repository-Hinweise ergänzen die Beschreibung mit nachvollziehbarer Herkunft.
6. **Sichtbare Unsicherheit:** Offene Entscheidungen, Widersprüche und fehlende Angaben bleiben erkennbar und führen zu konkreten Rückfragen.

Die Kategorien bilden einen ticketartübergreifenden Mindestbestand, kein starres For-

mular. Entscheidend sind Relevanz, Nachvollziehbarkeit und Prüfbarkeit, nicht Textmenge. Das kompakte Ausgabeschema des Prototyps bildet den Kern ab; sein Lösungsvorschlag bleibt ein veränderbarer Entwurf und keine Quelle fachlicher Wahrheit (vgl. Bettenburg et al., 2008, S. 308 f.; Lucassen et al., 2016, S. 386 f.; K C et al., 2025, S. 104–116).

9.1.2. Kontrollierte Integration eines KI-Assistenten

Die zweite Forschungsfrage lautet: *Wie kann ein KI-gestützter Assistent technisch so in einen GitLab-basierten Entwicklungsworkflow integriert werden, dass Analyse, Rückfragen, Vorschläge und Freigabe kontrolliert ablaufen?* Aus Architektur, Implementierung und Evaluation ergeben sich sechs notwendige Kontrollbausteine:

1. **Ereignisannahme:** Authentifizierung, Zeitstempel-, Typ- und ID-Prüfung, Bot-Filter, Aktivierungsregeln und Deduplizierung begrenzen zulässige Ereignisse.
2. **Kontextaufbereitung:** Die Anwendung wählt und begrenzt Issue-, Kommentar- und Repository-Daten und behandelt sie als nicht vertrauenswürdig.
3. **Modellkapselung:** Das Modell erhält nur den vorbereiteten Kontext und keine produktiven GitLab-Rechte.
4. **Antwortgrenze:** Parsing, Laufzeitschema, Herkunftsregeln und Kandidatenauswahl begrenzen den Modelloutput deterministisch.
5. **Konsistenter Zwischenstand:** Nur die vollständig sichtbare und danach freigegebene Revision ist veröffentlichtbar.
6. **Fehlersichere Änderung:** Ein autorisierter Publish-Schritt ändert zuerst das Issue; nachgeordnetes Löschen von Bot-Komentaren darf den Ausgangsstand bei Fehlern nicht zerstören.

Kontrolle entsteht durch die deterministische Anwendungshülle, nicht durch den Prompt allein. Der Prototyp demonstriert wesentliche Bausteine; Tabelle 7.1 weist jedoch offene Muss-Anforderungen an Sicherheits- und Integritätsgrenzen aus. Das Architekturprinzip ist damit als Zielmodell tragfähig, der kontrollierte Gesamtworkflow im untersuchten

Stand aber nicht nachgewiesen. Fachliche Informationsqualität und technische Prozessintegrität sind gemeinsam erforderlich.

9.2. Wissenschaftlicher und praktischer Beitrag

Die Arbeit leistet drei Beiträge: ein aus Literatur abgeleitetes Qualitätsmodell, ein Architekturmodell mit überprüfbaren Übergängen zwischen Ereignis, Modellantwort, Vorschau und Issue-Änderung sowie eine ausführbare Referenzimplementierung mit dokumentiertem Nachweisstand. Formale Struktur, Faktentreue und praktische Nutzbarkeit werden dabei getrennt. Die revisionsgebundene Freigabe macht eine häufig verdeckte Integritätsanforderung explizit.

Die Aussagekraft bleibt auf das untersuchte Projekt begrenzt. Die Beiträge liegen in Operationalisierung, Architektur und prüfbarer Bestandsaufnahme; Effektgrößen sind nicht Gegenstand des Nachweises.

9.3. Priorisierter Ausblick

Die weitere Arbeit muss sachlichen Abhängigkeiten folgen: Zuerst ist technische Schnittstellenkonformität herzustellen, danach kann semantische Qualität belastbar untersucht werden, und erst auf dieser Grundlage lässt sich praktischer Prozessnutzen bewerten. Tabelle 9.1 verdichtet diese Reihenfolge zu drei Entwicklungs- und Evaluationsstufen.

Tabelle 9.1: Priorisierte Roadmap vom Proof of Concept zur belastbaren Evaluation

Stufe	Ziel und Maßnahmen	Eintritts- beziehungsweise Erfolgskriterium
1: Schnittstellen- und Prozess- sicherheit	Signaturvorgaben von GitLab, manuelle Zugänge, IDs, Pfade, Revisionen, Publish-Reihenfolge und Idempotenz absichern	Positive und negative Integrationstests belegen sichere Eingangs-, Freigabe- und Fehlerpfade
2: Semantische Evaluation und Governance	Geschichtetes Ticketkorpus; Vergleich mit manueller und deterministischer Aufbereitung; wiederholte Läufe und unabhängige Bewertungen	Faktentreue, Kontextdeckung, Rückfragequalität, Prüfbarkeit und Korrekturaufwand sind vorab operationalisiert
3: Betrieb und Erweiterung	Queue, Monitoring und Mehrinstanzbetrieb; danach Anhänge, Profile und weitere Ereignisse	Mindeststandard bleibt unter Last und Teilausfällen erhalten; Erweiterungen besitzen eigene Sicherheits- und Regressionstests

Die Eintrittslogik lautet: technische Schnittstellenkonformität vor semantischer Qualität und diese vor praktischem Prozessnutzen. Andernfalls lassen sich Modell- und Prozessfehler nicht trennen. Erweiterungen müssen dieselben Herkunfts-, Berechtigungs- und Freigabegrenzen wie der Kernprozess einhalten.

9.4. Schlussfolgerung

Die zentrale Schlussfolgerung lautet: Generative Modelle können Ticketinhalte strukturieren, doch Verantwortlichkeit und Änderungsrecht verbleiben bei einer deterministischen, revisionsgebundenen Prozesshülle. Die Roadmap übersetzt diese Trennung in überprüfbare nächste Schritte: zuerst Schnittstellen und Zustände, dann semantische Güte und anschließend betriebliche Wirkung.

Quellen

Bashir, Sarmad; Ferrari, Alessio; Khan, Abbas; Strandberg, Per Erik; Haider, Zulqarnain; Saadatmand, Mehrdad; Bohlin, Markus 2025. „Requirements Ambiguity Detection and Explanation with LLMs: An Industrial Study“, in *2025 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, S. 620–631. IEEE. <https://doi.org/10.1109/ICSME64153.2025.00063>.

Bass, Len; Clements, Paul; Kazman, Rick 2021. *Software Architecture in Practice*. 4. Aufl. Boston: Addison-Wesley.

Bettenburg, Nicolas; Just, Sascha; Schröter, Adrian; Weiss, Cathrin; Premraj, Rahul; Zimmermann, Thomas 2008. „What Makes a Good Bug Report?“, in *Proceedings of the 16th ACM SIGSOFT International Symposium on Foundations of Software Engineering*, S. 308–318. New York: ACM. <https://doi.org/10.1145/1453101.1453146>.

Bo, Lili; Ji, Wangjie; Sun, Xiaobing; Zhang, Ting; Wu, Xiaoxue; Wei, Ying 2024. „ChatBR: Automated Assessment and Improvement of Bug Report Quality Using ChatGPT“, in *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, S. 1472–1483. New York: ACM. <https://doi.org/10.1145/3691620.3695518>.

Breu, Silvia; Premraj, Rahul; Sillito, Jonathan; Zimmermann, Thomas 2010. „Information Needs in Bug Reports: Improving Cooperation Between Developers and Users“, in *Proceedings of the 2010 ACM Conference on Computer Supported Cooperative Work*, S. 301–310. New York: ACM. <https://doi.org/10.1145/1718918.1718973>.

Brown, Tom B. et al. 2020. „Language Models Are Few-Shot Learners“, in *Advances in Neural Information Processing Systems* 33, S. 1877–1901. <https://arxiv.org/abs/2005.14165>.

Chaparro, Oscar; Lu, Jing; Zampetti, Fiorella; Moreno, Laura; Di Penta, Massimiliano; Marcus, Andrian; Bavota, Gabriele; Ng, Vincent 2017. „Detecting Missing Information in Bug Descriptions“, in *Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering*, S. 396–407. New York: ACM. <https://doi.org/10.1145/3106237.3106285>.

Easterbrook, Steve; Singer, Janice; Storey, Margaret-Anne; Damian, Daniela 2008. „Selecting Empirical Methods for Software Engineering Research“, in Shull, Forrest;

Singer, Janice; Sjøberg, Dag I. K. (Hrsg.): *Guide to Advanced Empirical Software Engineering*, S. 285–311. London: Springer. https://doi.org/10.1007/978-1-84800-044-5_11.

Express Documentation o. J. *Express: Node.js Web Application Framework*. <https://expressjs.com/> (Zugriff vom 31.07.2026).

Frattoni, Julian; Montgomery, Lloyd; Fischbach, Jannik; Mendez, Daniel; Fucci, Davide; Unterkalmsteiner, Michael 2023. „Requirements Quality Research: A Harmonized Theory, Evaluation, and Roadmap“, in *Requirements Engineering* 28, 4, S. 507–520. <https://doi.org/10.1007/s00766-023-00405-y>.

GitLab Documentation o. J.a. *Webhooks*. <https://docs.gitlab.com/user/project/integrations/webhooks/> (Zugriff vom 31.07.2026).

GitLab Documentation o. J.b. *Webhook Events*. https://docs.gitlab.com/user/project/integrations/webhook_events/ (Zugriff vom 31.07.2026).

GitLab Documentation o. J.c. *Issues API*. <https://docs.gitlab.com/api/issues/> (Zugriff vom 31.07.2026).

GitLab Documentation o. J.d. *Notes API*. <https://docs.gitlab.com/api/notes/> (Zugriff vom 31.07.2026).

Hohpe, Gregor; Woolf, Bobby 2003. *Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions*. Boston: Addison-Wesley. <https://www.enterpriseintegrationpatterns.com/>.

Huang, Lei et al. 2023. „A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions“. arXiv:2311.05232. <https://doi.org/10.48550/arXiv.2311.05232>.

Ibtasham, Md Saleh; Bashir, Sarmad; Abbas, Muhammad; Haider, Zulqarnain; Saadatmand, Mehrdad; Cicchetti, Antonio 2025. „ReqRAG: Enhancing Software Release Management through Retrieval-Augmented LLMs: An Industrial Study“, in Hess, Anne; Susi, Angelo (Hrsg.): *Requirements Engineering: Foundation for Software Quality. 31st International Working Conference, REFSQ 2025, Lecture Notes in Computer Science 15588*, S. 277–292. Cham: Springer. https://doi.org/10.1007/978-3-031-88531-0_20.

Inayat, Irum; Salim, Siti Salwah; Marczak, Sabrina; Daneva, Maya; Shamshirband,

Shahaboddin 2015. „A Systematic Literature Review on Agile Requirements Engineering Practices and Challenges“, in *Computers in Human Behavior* 51, S. 915–929. <https://doi.org/10.1016/j.chb.2014.10.046>.

ISO/IEC 2023. *ISO/IEC 25010:2023. Systems and Software Engineering – Systems and Software Quality Requirements and Evaluation (SQuaRE) – Product Quality Model*. Öffentliche Katalogseite. <https://www.iso.org/standard/78176.html> (Zugriff vom 31.07.2026).

ISO/IEC/IEEE 2018. *ISO/IEC/IEEE 29148:2018. Systems and Software Engineering – Life Cycle Processes – Requirements Engineering*. Öffentliche Katalogseite. <https://www.iso.org/standard/72089.html> (Zugriff vom 31.07.2026).

K C, Pragyan; Ghandiparsi, Rambod; Herron, Thomas; Heaps, John; Hosseini, Mitra Bokaei 2025. „Demystifying Feature Requests: Leveraging LLMs to Refine Feature Requests in Open-Source Software“, in *2025 IEEE 33rd International Requirements Engineering Conference (RE)*, S. 104–116. IEEE. <https://doi.org/10.1109/RE63999.2025.00020>.

Krishna, Madhava; Gaur, Bhagesh; Verma, Arsh; Jalote, Pankaj 2024. „Using LLMs in Software Requirements Specifications: An Empirical Evaluation“, in *2024 IEEE 32nd International Requirements Engineering Conference (RE)*, S. 475–483. IEEE. <https://doi.org/10.1109/RE59067.2024.00056>.

Liu, Yi et al. 2023. „Prompt Injection Attack against LLM-Integrated Applications“. arXiv:2306.05499. <https://doi.org/10.48550/arXiv.2306.05499>.

Lubos, Sebastian; Felfernig, Alexander; Tran, Thi Ngoc Trang; Garber, Damian; El Mansi, Merfat; Erdeniz, Seda Polat; Le, Viet-Man 2024. „Leveraging LLMs for the Quality Assurance of Software Requirements“, in *2024 IEEE 32nd International Requirements Engineering Conference (RE)*, S. 389–397. IEEE. <https://doi.org/10.1109/RE59067.2024.00046>.

Lucassen, Garm; Dalpiaz, Fabiano; van der Werf, Jan Martijn E. M.; Brinkkemper, Sjaak 2016. „Improving Agile Requirements: The Quality User Story Framework and Tool“, in *Requirements Engineering* 21, 3, S. 383–403. <https://doi.org/10.1007/s00766-016-0250-x>.

Mahbub, Taslim; Dghaym, Dana; Shankarnarayanan, Aadith; Syed, Taufiq; Shapsough, Salsabeel; Zualkernan, Imran 2024. „Can GPT-4 Aid in Detec-

ting Ambiguities, Inconsistencies, and Incompleteness in Requirements Analysis? A Comprehensive Case Study“, in *IEEE Access* 12, S. 171972–171992. <https://doi.org/10.1109/ACCESS.2024.3464242>.

National Institute of Standards and Technology 2024. *Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile*. NIST AI 600-1. Gaithersburg, MD: National Institute of Standards and Technology. <https://doi.org/10.6028/NIST.AI.600-1>.

Node.js Documentation o. J.a. *Node.js*. <https://nodejs.org/en> (Zugriff vom 31.07.2026).

Node.js Documentation o. J.b. *HTTP*. <https://nodejs.org/api/http.html> (Zugriff vom 31.07.2026).

OpenCode Documentation o. J.a. *Agents*. <https://opencode.ai/docs/agents/> (Zugriff vom 31.07.2026).

OpenCode Documentation o. J.b. *CLI*. <https://opencode.ai/docs/cli/> (Zugriff vom 31.07.2026).

OWASP 2025a. *LLM01:2025 Prompt Injection*. OWASP Top 10 for LLM Applications 2025. <https://genai.owasp.org/llmrisk/llm01-prompt-injection/> (Zugriff vom 31.07.2026).

OWASP 2025b. *LLM06:2025 Excessive Agency*. OWASP Top 10 for LLM Applications 2025. <https://genai.owasp.org/llmrisk/llm062025-excessive-agency/> (Zugriff vom 31.07.2026).

Paiva, Guilherme; Canedo, Edna Dias; Rocha Filho, Geraldo Pereira 2026. „From Issue Titles to Requirements: An Empirical Study of Large Language Models and Prompt Engineering Strategies“, in *Requirements Engineering* 31, Artikel 7. <https://doi.org/10.1007/s00766-026-00462-z>.

Saleem, Summra; Asim, Muhammad Nabeel; Van Elst, Ludger; Dengel, Andreas 2025. „Generative Language Models Potential for Requirement Engineering Applications: Insights into Current Strengths and Limitations“, in *Complex & Intelligent Systems* 11, 6, Artikel 278. <https://doi.org/10.1007/s40747-024-01707-6>.

Schick, Timo et al. 2023. „Toolformer: Language Models Can Teach Themselves to

Use Tools“, in *Advances in Neural Information Processing Systems* 36, S. 68539–68551. <https://arxiv.org/abs/2302.04761>.

Song, Yang; Chaparro, Oscar 2020. „BEE: A Tool for Structuring and Analyzing Bug Reports“, in *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, S. 1551–1555. New York: ACM. <https://doi.org/10.1145/3368089.3417928>.

Tizard, James; Devine, Peter; Wang, Hechen; Blincoe, Kelly 2023. „A Software Requirements Ecosystem: Linking Forum, Issue Tracker, and FAQs for Requirements Management“, in *IEEE Transactions on Software Engineering* 49, 4, S. 2381–2393. <https://doi.org/10.1109/TSE.2022.3219458>.

TypeScript Documentation o. J. *The TypeScript Handbook*. <https://www.typescriptlang.org/docs/handbook/intro.html> (Zugriff vom 31.07.2026).

Werner, Nico 2026. *Dev-Assist*. Interner Projektstand, Git-Commit d179eb22beb0a59006869a3acc7829f3070fc678 vom 26.07.2026, geprüft am 31.07.2026; Artefaktübersicht siehe Tabelle 6.2.

Xi, Zhiheng et al. 2023. „The Rise and Potential of Large Language Model Based Agents: A Survey“. arXiv:2309.07864. <https://doi.org/10.48550/arXiv.2309.07864>.

Yao, Shunyu et al. 2023. „ReAct: Synergizing Reasoning and Acting in Language Models“, in *International Conference on Learning Representations 2023*. https://openreview.net/forum?id=WE_vluYUL-X.

Zhao, Wayne Xin et al. 2023. „A Survey of Large Language Models“. arXiv:2303.18223. <https://doi.org/10.48550/arXiv.2303.18223>.

A. Illustrierter Ticketverlauf

Der folgende Verlauf ist ein konstruiertes Illustrationsbeispiel. Er verwendet das Ausgabeschema und die Formatter-Logik des untersuchten Prototyps, jedoch keine betrieblichen Originaldaten. Er belegt nicht die semantische Qualität realer Modellantworten.

Ausgangsticket

Titel: Login funktioniert nicht

Beschreibung: Der Login auf der internen Oberfläche funktioniert nicht richtig.

@dev-assist

Fehlend sind beobachtbares Ist-Verhalten, erwartetes Ergebnis, betroffene Rolle, Reproduktion und prüfbare Akzeptanzkriterien. Nach FA-6 müssten daraus konkrete Rückfragen entstehen.

Sichtbarer Klärungskommentar

Im untersuchten Stand erzeugt der Processor bei mindestens zwei substanziellen offenen Fragen einen Klärungskommentar. Eine schematisch gültige, fachlich unvollständige Analyse könnte etwa zu folgenden Fragen führen:

- Welches beobachtbare Verhalten tritt beim Anmelden auf, und in welchem Schritt bricht der Vorgang ab?
- Welches Ergebnis gilt als erfolgreich: Sitzung, Weiterleitung oder Fehlermeldung?
- Welche Nutzerrolle und welcher Zugangsweg sind betroffen?

Lokal wird in diesem Fall dennoch der vollständige Analysekontext in `context.md` gespeichert. Genau diese Differenz zwischen sichtbarer Klärung und gespeichertem Vorschlag ist die in Kapitel 7 beschriebene Lücke zwischen Vorschau und gespeichertem Stand.

Strukturierter Vorschlag nach Klärung

Nach beantworteten Rückfragen kann eine erneute Analyse dasselbe Schema füllen. Ein verdichteter, weiterhin fiktiver Entwurf sähe etwa so aus:

Titel: Anmeldefehler nach Passworteingabe reproduzierbar machen und beheben

Description. Fachliche Nutzerinnen und Nutzer können sich an der internen Oberfläche nicht anmelden. Nach korrekter Passworteingabe bleibt die Maske stehen; es erscheint keine Fehlermeldung und keine Sitzung. Ziel ist ein nachvollziehbarer Anmeldevorgang mit eindeutigem Erfolg oder einer verständlichen Ablehnung.

Acceptance Criteria.

- Bei gültigen Zugangsdaten entsteht eine Sitzung und die Startseite wird angezeigt.
- Bei ungültigen Zugangsdaten erscheint eine konkrete Fehlermeldung, ohne dass eine Sitzung entsteht.
- Der Fehlerfall ist mit Rolle, Browser und Uhrzeit reproduzierbar dokumentiert.

Technical Context. Die Beschreibung nennt weder HTTP-Status noch Serverlog. Technische Ursache und betroffene Komponente sind daher nicht belegt.

Proposed Solution. Zunächst den Fehler reproduzieren und Logs der Anmeldegrenze sichern. Anschließend Ist- und Soll-Verhalten in der Oberfläche angleichen und die Fehlermeldung prüfbar machen.

Open Questions. Keine offenen Produktfragen, sofern Reproduktion und Erfolgskriterium bestätigt sind.

Erst @dev-assist publish würde diesen Stand in das Issue schreiben. Im untersuchten Stand prüft der Publisher nicht, ob genau diese Revision vollständig sichtbar und freigegeben war.

B. Dokumentierter Prüfstand

Die folgende Übersicht wiederholt den in Kapitel 6 dokumentierten Lauf in einer für die Prüfung nachvollziehbaren Kurzform.

Tabelle B.1: Kurzprotokoll des dokumentierten Build- und Testlaufs

Größe	Angabe
Datum und Umgebung	31.07.2026, Windows, Node.js v24.11.0, npm 11.6.1
Untersuchter Stand	Git-Commit d179eb22beb0a59006869a3acc7829f3070fc678
Build	npm run build, ohne TypeScript-Fehler
Tests	npm test: 60 von 60 Tests in 15 Suiten bestanden, ca. 0,6 Sekunden
Coverage	nicht konfiguriert
Ende-zu-Ende gegen GitLab	nicht ausgeführt

Die Tests prüfen vor allem Parser, Schema, Formatter, ausgewählte Laufzeitpfade und Aufbereitungslogik. Sie ersetzen weder einen realen Webhook-Lauf noch eine fachliche Bewertung von Modellantworten.

C. Ausgabeschema des Prototyps

Das implementierte Ausgabeschema akzeptiert ausschließlich die folgenden Felder. Unbekannte Felder oder falsche Datentypen werden abgelehnt.

Tabelle C.1: Felder des kompakten Ausgabeschemas

Feld	Typ	Zweck
title	Zeichenkette	prägnanter Issue-Titel
description	Zeichenkettenliste	Ziel, Nutzen, Verhalten und Scope in Absätzen
acceptanceCriteria	Zeichenkettenliste	beobachtbare Prüfkriterien
technicalContext	Zeichenkettenliste	belegte technische Hinweise, Logs, Annahmen
proposedSolution	Zeichenkettenliste	grobe Umsetzungsschritte ohne fachliche Erfindung
openQuestions	Zeichenkettenliste	unbeantwortete fachliche Fragen

Quellenbasis, Risiken und Validierungsschritte des fachlichen Zielmodells aus Kapitel 3 besitzen im kompakten Ausgabeschema keine eigenen Felder. Sie können nur innerhalb der allgemeinen Listen erscheinen oder entfallen.